

ÜNİTE 5

ÇEKİŞMELİ ARAMA

Adversarial Search

Türkçe çeviri + ayrıntılı açıklamalar + görseller + örnekler +
formüller + alıştırma çözümleri

Yapay Zekâya Giriş - Ders Çalışma Notu

Kaynak metin: kullanıcı tarafından sağlanan AIMA 3. baskı Ünite 5 bölümleri ve
alıştırmaları.

Türkçe açıklama ve öğretim örnekleri bu not için hazırlanmıştır.

İçindekiler

1	5. Çekişmeli Arama - Adversarial Search	2
2	5.1 Oyunlar - Games	2
2.1	Çok ajanlı ve rekabetçi ortamlar	2
2.2	Bir oyunun biçimsel tanımı	3
2.3	Oyun ağacı ve arama ağacı	3
3	5.2 Oyunlarda Optimal Kararlar	4
3.1	Minimax değeri	4
3.1.1	Şekil 5.2'nin adım adım çözümü	5
3.2	Minimax algoritması	6
3.3	Çok oyunculu oyunlar	6
4	5.3 Alpha-Beta Budama	7
4.1	Şekil 5.5'in adım adım açıklaması	8
4.2	Alpha ve Beta sınırları	9
4.3	Hamle sıralaması	10
4.4	Iterative deepening, killer move ve transposition table	11
5	5.4 Eksik Gerçek Zamanlı Kararlar	11
5.1	Heuristic Minimax	11
5.2	5.4.1 Değerlendirme fonksiyonları	11
5.2.1	Beklenen değer örneği	12
5.2.2	Taş değeri ve ağırlıklı doğrusal fonksiyon	12
5.3	5.4.2 Aramayı kesme	13
5.3.1	Quiescence search	13
5.3.2	Horizon effect	14
5.4	5.4.3 Forward pruning	14
5.5	5.4.4 Search versus lookup	15
6	5.5 Stokastik Oyunlar	15
6.1	Expectiminimax	16
6.2	5.5.1 Şans oyunlarında değerlendirme fonksiyonları	17
6.2.1	Chance pruning ve Monte Carlo rollout	17

7 5.6 Kısmen Gözlemlenebilir Oyunlar	18
7.1 5.6.1 Kriegspiel: kısmen gözlemlenebilir satranç	18
7.1.1 Garantili, olasılıksal ve tesadüfi mat	18
7.2 5.6.2 Kart oyunları	19
7.2.1 Averaging over clairvoyance problemi	20
8 5.7 Döneminin En Gelişmiş Oyun Programları	20
8.1 Satranç	20
8.2 Checkers, Othello, Backgammon	21
8.3 Go	21
8.4 Bridge ve Scrabble	21
9 5.8 Alternatif Yaklaşımlar	21
9.1 Düşüm genişletmenin faydası ve metareasoning	22
9.2 Hedef yönelimli planlama	22
105.9 Öğretim Amaçlı Bölüm Özeti	23
10.1 Hızlı formül sayfası	23
11 Alıştırmalar 5.1-5.22 - Türkçe Çeviri ve Çözümler	25
11.15.1 Rakibin hamlesini bilen kahin	25
11.25.2 İki adet 8-puzzle	25
11.35.3 Takip-kaçış oyunu	26
11.45.4 Stokastik oyun modeli tasarlama	27
11.55.5 Gerçek zamanlı çok oyunculu ortam	27
11.65.6 Sürekli fiziksel oyunlar	28
11.75.7 Suboptimal rakibe karşı minimax	28
11.85.8 Dört karelik oyun ve döngüler	28
11.95.9 Tic-tac-toe evaluation alıştırmaları	29
11.10.10 Genelleştirilmiş tic-tac-toe	30
11.15.11 Genel oyun oynayan program	30
11.18.12 İki oyunculu sıfır toplamli olmayan oyunlar	30
11.13.13 Alpha-Beta doğruluk ispatı	31
11.14.14 Alpha-Beta'nın ideal karmaşıklığı	32
11.15.15 Transposition table kapasitesi	32
11.16.16 Chance node'larda budama	33

11.17.17 Expectiminimax ve *-Alpha-Beta	34
11.18.18 Pozitif doğrusal dönüşüm	34
11.19.19 Zar dizilerini önceden örnekleme	34
11.20.20 Max ve Expectimax ağaçlarında budama	35
11.21.21 Doğru / yanlış	35
11.22.22 Chance ve eksik bilginin birlikte etkisi	35

12 Kavram Sözlüğü**37**

5. Çekişmeli Arama - Adversarial Search

Bu bölümde, başka ajanların da bizim aleyhimize plan yaptığı ortamlarda ileriye dönük karar vermenin neden normal aramadan daha zor olduğu incelenir. Normal bir rota bulma probleminde yol ağı bizim seçimimize karşı bilinçli biçimde tepki vermez. Buna karşılık satranç, dama veya tic-tac-toe gibi oyunlarda rakip, bizim faydamızı azaltacak hamleler seçer. Dolayısıyla bir hamlenin iyi olup olmadığını değerlendirirken yalnızca kendi eylemlerimizi değil, rakibin verebileceği en iyi cevapları da hesaba katmak gerekir.

Bölümün ana sorusu

Bir ajan, rakibinin de rasyonel biçimde kendisine karşı oynadığı bir ortamda hangi hamleyi seçmelidir? Oyun ağacı çok büyükse, zaman sınırlıysa, rastlantı varsa veya tüm bilgiler görülemiyorsa bu karar nasıl yaklaşılaştırılır?

5.1 Oyunlar - Games

Çok ajanlı ve rekabetçi ortamlar

Çok ajanlı bir ortamda her ajan, diğer ajanların eylemlerinin kendi başarısını nasıl etkileyeceğini düşünmek zorundadır. Rekabetçi ortamlarda ajanların amaçları çatıştığı için problem **çekişmeli arama** (*adversarial search*) problemine dönüşür. Matematiksel oyun teorisi işbirlikçi veya rekabetçi çok ajanlı durumların tamamını oyun olarak ele alabilir. Yapay zekâ literatüründe ise başlangıç noktası çoğunlukla daha özel bir sınıftır: deterministik, sıra tabanlı, iki oyunculu, sıfır toplam ve tam bilgili oyunlar.

Temel oyun özellikleri

- **Deterministik:** Bir hamlenin sonucu rastlantısal değildir.
- **Sıra tabanlı:** Oyuncular sırayla hamle yapar.
- **İki oyunculu:** Başlangıçta iki karar verici ele alınır: MAX ve MIN.
- **Tam bilgili:** Mevcut oyun durumu her iki oyuncu tarafından da görülebilir.
- **Sıfır toplam:** Oyuncuların çıkarları zıttır; MAX için iyi olan sonuç MIN için kötüdür.

Satranç bu çerçeve için klasik bir örnektir. Bir taraf kazandığında diğeri kaybeder; taktadaki bütün taşlar görünür ve hamlelerin sonucu belirgindir. Poker ise tam bilgili değildir; tavla ise zar nedeniyle deterministik değildir.

Oyunlar yapay zekâ için çekicidir çünkü kuralları ve durumları açık biçimde temsil edilebilir. Buna karşın arama uzayları çok büyüktür. Satrançta ortalama dallanma faktörü yaklaşık 35'tir ve uzun bir oyunda yüzlerce yarım hamle oluşabilir. Bu nedenle bütün oyun ağacını üretmek fiziksel olarak mümkün değildir. Bir oyun programı, optimal kararın eksiksiz

hesabı mümkün değilken dahi iyi bir karar vermek zorundadır.

Bir oyunun biçimsel tanımı

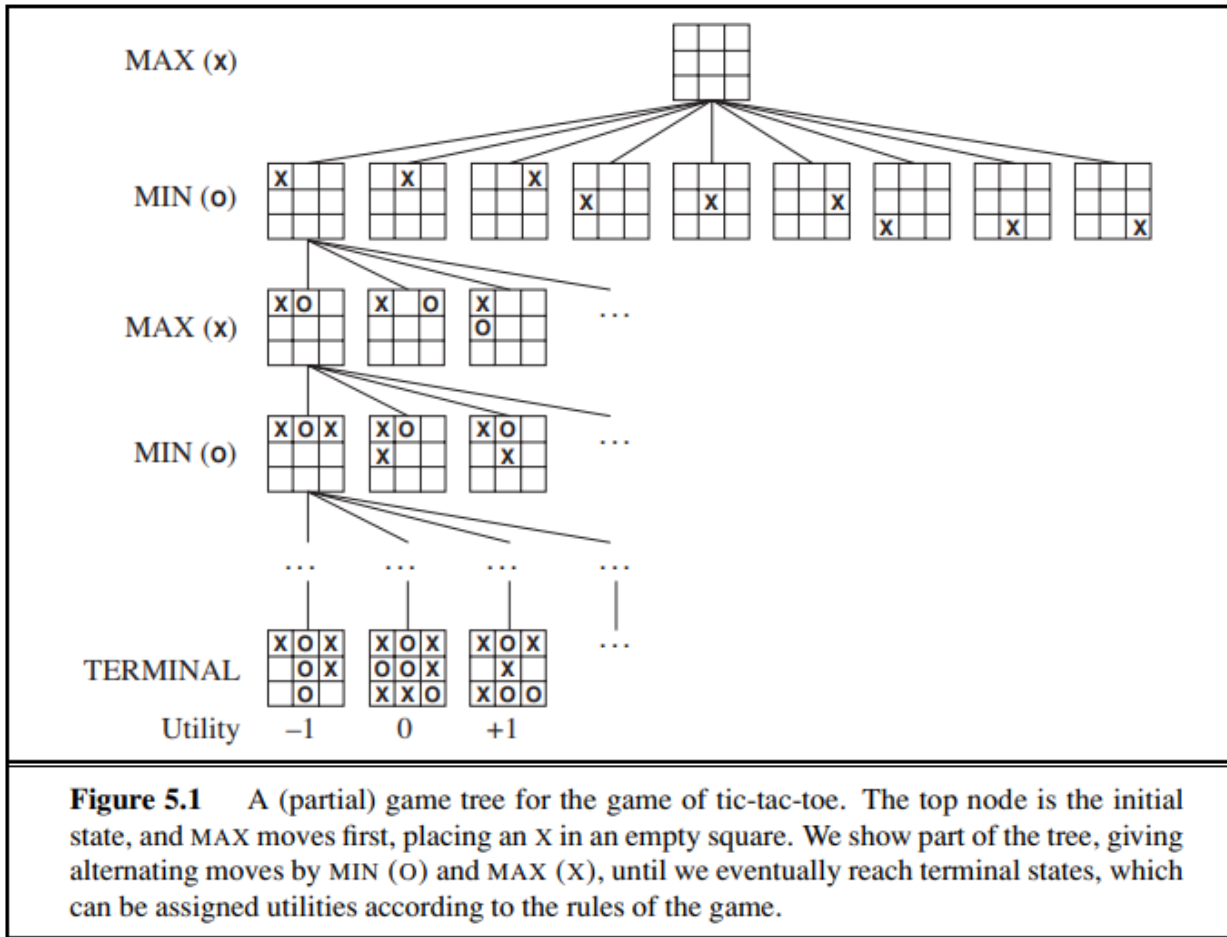
İki oyunculu bir oyun aşağıdaki bileşenlerle tanımlanabilir:

- S_0 : Oyunun başlangıç durumu.
- $PLAYER(s)$: Durum s 'de sıranın hangi oyuncuda olduğunu belirtir.
- $ACTIONS(s)$: Durum s 'deki yasal hamleleri verir.
- $RESULT(s, a)$: a hamlesi uygulandığında oluşan yeni durumu verir.
- $TERMINAL-TEST(s)$: Oyunun bitip bitmediğini sınar.
- $UTILITY(s, p)$: Terminal durumun oyuncu p açısından sayısal değerini verir.

Örneğin basit bir sıfır toplamlı oyunda MAX için kazanç $+1$, beraberlik 0 , kayıp -1 olarak gösterilebilir. MIN aynı sonuca ters yönden bakar.

Oyun ağacı ve arama ağacı

S_0 , $ACTIONS$ ve $RESULT$ birlikte tüm olası hamle dizilerini içeren **oyun ağacını** (*game tree*) tanımlar. Düğümler oyun durumlarını, kenarlar hamleleri temsil eder. Fakat gerçek bir program çoğu zaman bu ağacın yalnızca küçük bir bölümünü inceler; gerçekten oluşturulan bu alt yapıya **arama ağacı** (*search tree*) denir.



Şekil 1: Tic-tac-toe için kısmi oyun ağacı. MAX, X taşını; MIN, O taşını yerleştirir. Terminal durumlar MAX açısından -1, 0 veya +1 fayda ile değerlendirilir.

Örnek - Tic-tac-toe ağacını okumak

Boş tahtada MAX'ın dokuz farklı ilk hamlesi vardır. MAX bir kareye X koyduktan sonra MIN kalan sekiz kareden birine O koyabilir. Sonraki katmanda tekrar MAX oynar. Bu dönüşümlü yapı oyun bitene kadar sürer. Yapraklardaki fayda değerleri MAX açısından yazıldığı için MAX yüksek, MIN düşük değeri tercih eder.

5.2 Oyunlarda Optimal Kararlar

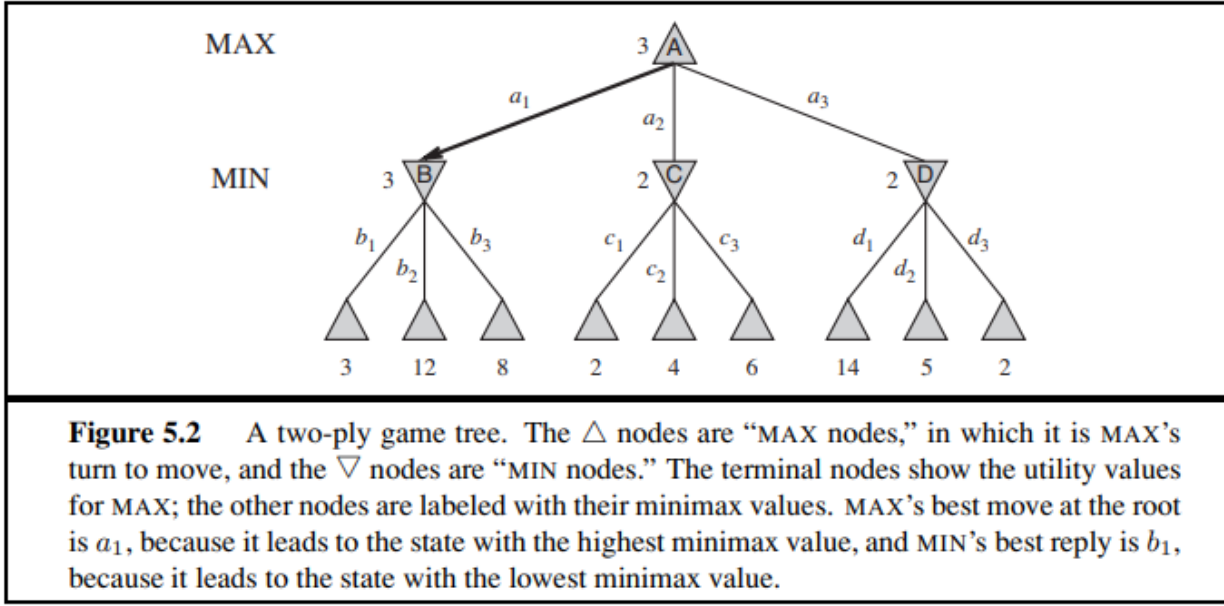
Normal aramada çözüm tek bir eylem dizisi olabilir. Rakipli bir oyunda ise rakibin hangi cevabı vereceği bilinmediğinden MAX'ın bir **stratejiye** (*strategy*) ihtiyacı vardır. Strateji, rakibin her olası cevabından sonra ne yapılacağını belirleyen koşullu bir plandır.

Minimax değeri

Bir düğümün **minimax değeri** (*minimax value*), o durumdan sonra iki oyuncunun da kusursuz oynadığı varsayımı altında MAX'ın elde edeceği faydadır. Terminal bir düğümün minimax değeri doğrudan utility değeridir. MAX düğümü en yüksek, MIN düğümü en

düşük çocuk değerini seçer:

$$MINIMAX(s) = \begin{cases} UTILITY(s), & TERMINAL-TEST(s) \\ \max_{a \in ACTIONS(s)} MINIMAX(RESULT(s, a)), & PLAYER(s) = MAX \\ \min_{a \in ACTIONS(s)} MINIMAX(RESULT(s, a)), & PLAYER(s) = MIN \end{cases}$$



Şekil 2: İki ply’lik oyun ağacı. MAX kökte en yüksek geri taşınmış değeri; MIN kendi düğümünde en düşük değeri seçer.

Şekil 5.2’nin adım adım çözümü

B düğümü bir MIN düğümüdür:

$$V(B) = \min(3, 12, 8) = 3.$$

C için:

$$V(C) = \min(2, 4, 6) = 2.$$

D için:

$$V(D) = \min(14, 5, 2) = 2.$$

Kök A bir MAX düğümüdür:

$$V(A) = \max(3, 2, 2) = 3.$$

Dolayısıyla MAX’in optimal ilk hamlesi a_1 ’dir.

Minimax sezgisi

MAX, “Rakibim bana verilebilecek en kötü cevabı verecek; buna rağmen hangi ilk hamle bana en iyi garantiyi sağlar?” diye düşünür. Bu nedenle minimax, en kötü duruma karşı en iyi garantiyi seçen korumacı bir stratejidir.

Minimax algoritması

Minimax, oyun ağacını derinlik öncelikli biçimde terminal düğümlere kadar araştırır ve değerleri alttan üste doğru taşır.

Minimax sözde kodu

```

MINIMAX-DECISION(state):
    en yüksek MIN-VALUE sonucunu veren hamleyi seç

MAX-VALUE(state):
    eğer terminal ise UTILITY(state) döndür
    v = -sonsuz
    her hamle a için:
        v = max(v, MIN-VALUE(RESULT(state,a)))
    v döndür

MIN-VALUE(state):
    eğer terminal ise UTILITY(state) döndür
    v = +sonsuz
    her hamle a için:
        v = min(v, MAX-VALUE(RESULT(state,a)))
    v döndür

```

Dallanma faktörü b , maksimum derinlik m ise zaman karmaşıklığı:

$$O(b^m)$$

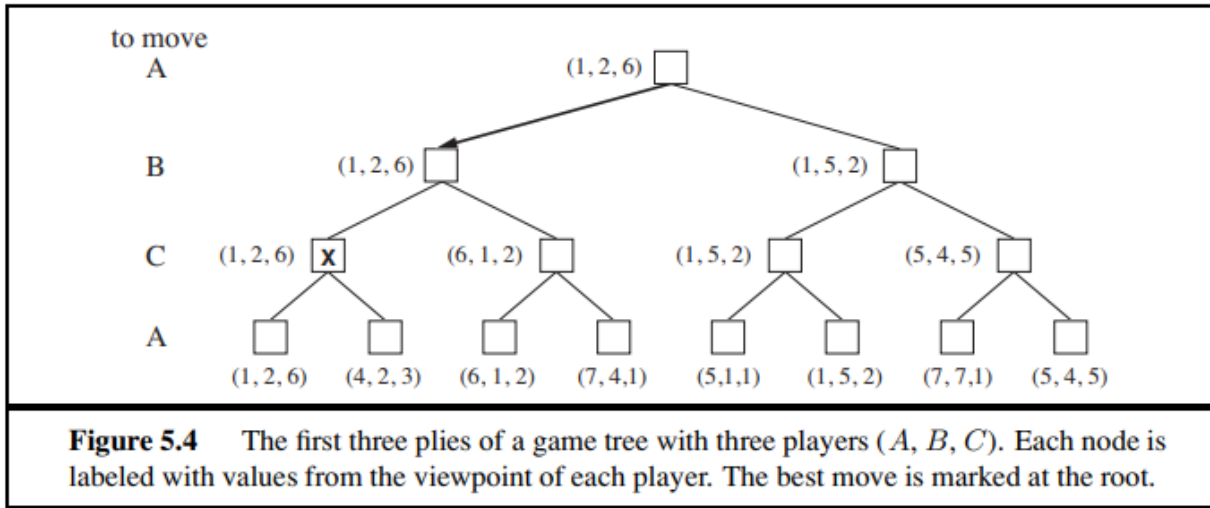
olur. Bu üstel büyüme gerçek oyunlarda temel darboğazdır. Eylemler birer birer üretilirse derinlik öncelikli uygulamanın bellek maliyeti yaklaşık $O(m)$ olabilir; bütün çocuklar birlikte tutulursa $O(bm)$ seviyesine çıkar.

Çok oyunculu oyunlar

İkiden fazla oyuncu olduğunda tek bir utility sayısı yeterli değildir. Üç oyunculu bir oyunda her terminal duruma:

$$(v_A, v_B, v_C)$$

biçiminde bir fayda vektörü atanabilir. Sırası gelen oyuncu kendi bileşenini en büyük yapan çocuğu seçer.



Şekil 3: Üç oyunculu oyunda her düğüm A , B ve C açısından bir fayda vektörü taşır. Sırası gelen oyuncu vektörün kendi bileşenini maksimize eder.

Şekilde X düğümünde sıra C oyuncusundadır. İki sonuç $(1, 2, 6)$ ve $(4, 2, 3)$ olduğuna göre C , kendi faydası 6 olduğu için ilkinin seçer. Bu nedenle X düğümüne geri taşınan vektör $(1, 2, 6)$ olur.

Çok oyunculu oyunlarda geçici ittifaklar da doğal biçimde ortaya çıkabilir. İki zayıf oyuncunun güçlü bir üçüncü oyuncuya karşı geçici olarak işbirliği yapması, tamamen bireysel çıkar maksimizasyonundan doğabilir. Sıfır toplamı olmayan iki oyunculu oyunlarda da ortak fayda sağlayan durumlar işbirliğini optimal hale getirebilir.

5.3 Alpha-Beta Budama

Minimax'ın temel sorunu $O(b^m)$ düğüm incelemesidir. **Alpha-Beta budama** (*alpha-beta pruning*), minimax ile aynı kararı üretirken sonucu değiştiremeyecek dalları araştırmadan atar.

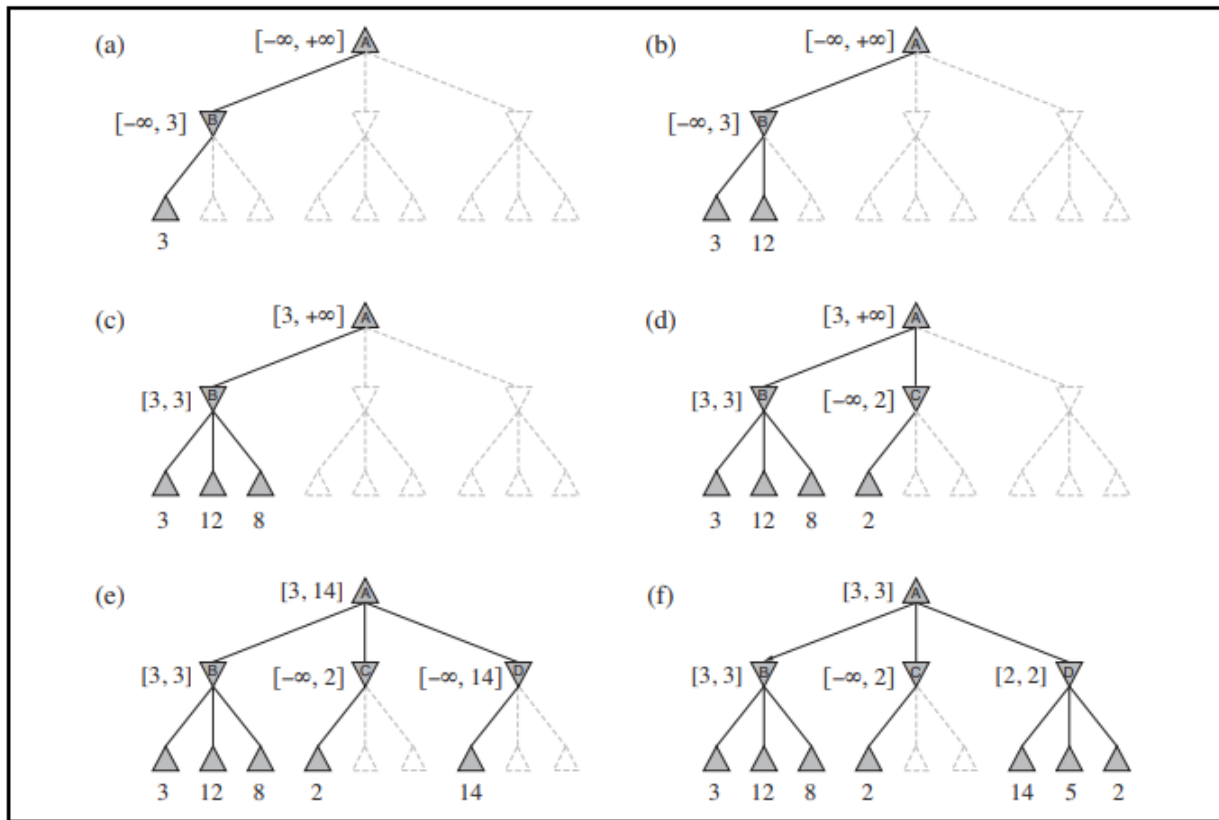


Figure 5.5 Stages in the calculation of the optimal decision for the game tree in Figure 5.2. At each point, we show the range of possible values for each node. (a) The first leaf below B has the value 3. Hence, B , which is a MIN node, has a value of *at most* 3. (b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still *at most* 3. (c) The third leaf below B has a value of 8; we have seen all B 's successor states, so the value of B is exactly 3. Now, we can infer that the value of the root is *at least* 3, because MAX has a choice worth 3 at the root. (d) The first leaf below C has the value 2. Hence, C , which is a MIN node, has a value of *at most* 2. But we know that B is worth 3, so MAX would never choose C . Therefore, there is no point in looking at the other successor states of C . This is an example of alpha-beta pruning. (e) The first leaf below D has the value 14, so D is worth *at most* 14. This is still higher than MAX's best alternative (i.e., 3), so we need to keep exploring D 's successor states. Notice also that we now have bounds on all of the successors of the root, so the root's value is also *at most* 14. (f) The second successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX's decision at the root is to move to B , giving a value of 3.

Şekil 4: Alpha-Beta budamanın Şekil 5.2'deki oyun ağacında adım adım uygulanması. Düğümlerdeki aralıklar, o ana kadar bilinen alt ve üst sınırları gösterir.

Şekil 5.5'in adım adım açıklaması

- (a) B altındaki ilk yaprak 3'tür. B bir MIN düğümü olduğundan $B \leq 3$.
- (b) İkinci yaprak 12 olsa da MIN zaten 3 seçeneğine sahip olduğu için $B \leq 3$ bilgisi değişmez.
- (c) Üçüncü yaprak 8'dir; artık bütün çocuklar görüldüğü için:

$$B = \min(3, 12, 8) = 3.$$

Kök MAX artık en az 3 elde edebileceğini bilir.

(d) C altındaki ilk yaprak 2'dir. C bir MIN düğümü olduğundan $C \leq 2$. Kök MAX'ın zaten 3 veren bir alternatifi bulunduğu için C hiçbir zaman seçilemez. Böylece C 'nin kalan iki çocuğu budanır.

(e) D 'nin ilk çocuğu 14'tür. $D \leq 14$ bilgisi henüz budama için yeterli değildir; çünkü 14, MAX'ın mevcut 3 garantisinden daha iyi olabilir.

(f) İkinci değer 5, üçüncü değer 2 gelir. Böylece $D = 2$ olur. Kök değeri $\max(3, 2, 2) = 3$ ve karar B dalıdır.

Örnek - Neden C 'nin iki yaprağına bakmaya gerek yok?

C 'nin bilinmeyen iki çocuğu x ve y olsun:

$$\max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)).$$

İkinci terim her zaman ≤ 2 'dir. Kökün elinde zaten 3 bulunduğu için x ve y hangi değer olursa olsun kök kararı değişmez.

Alpha ve Beta sınırları

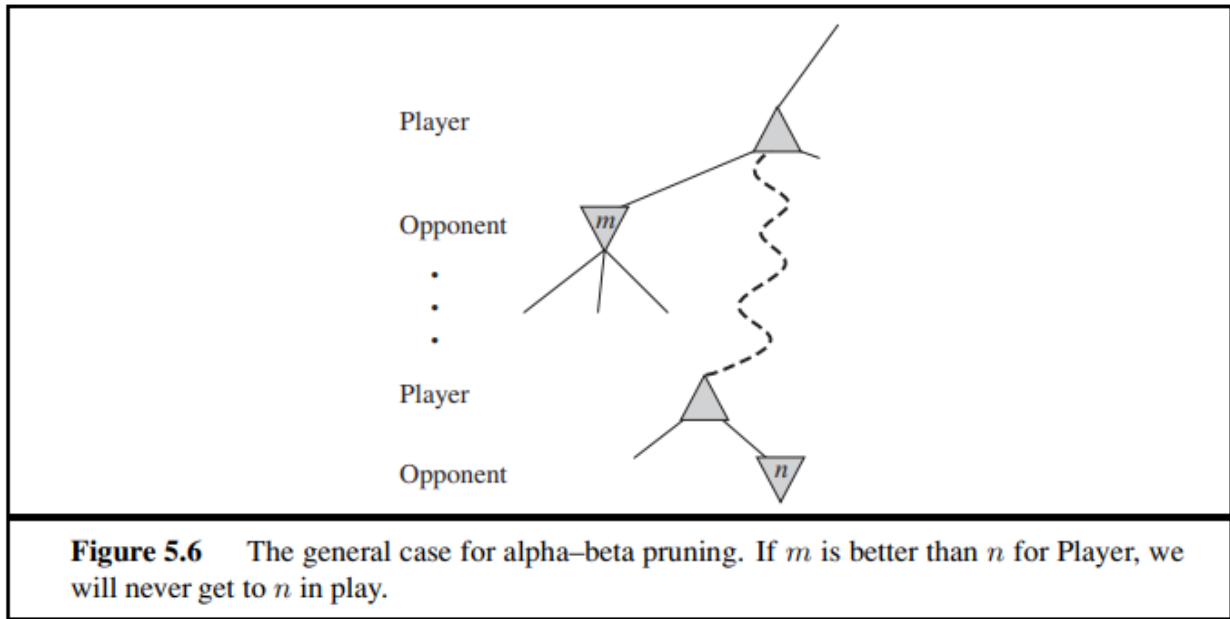
Alpha (α)

MAX'ın, kökten mevcut düğüme kadar olan yol üzerinde şimdiye kadar bulduğu en yüksek garanti değeridir. Başlangıçta $\alpha = -\infty$.

Beta (β)

MIN'in, kökten mevcut düğüme kadar olan yol üzerinde şimdiye kadar bulduğu en düşük garanti değeridir. Başlangıçta $\beta = +\infty$.

MAX düğümünde bir değer β 'ya ulaştığında veya onu geçtiğinde, MIN üstteki seçim noktasında bu dala izin vermeyeceği için kalan çocuklar budanabilir. MIN düğümünde değer α 'ya eşit veya daha düşük olduğunda, MAX'ın zaten daha iyi bir alternatifi olduğu için kalan çocuklar budanabilir.



Şekil 5: Alpha-Beta budamanın genel prensibi. Oyuncunun elinde m gibi daha iyi bir seçenek varsa ve n 'nin onu geçemeyeceği anlaşılmışsa n 'ye giden dal gerçek oyunda seçilmeyecektir.

Alpha-Beta sözde kodu

```

MAX-VALUE(state, alpha, beta):
    eğer terminal ise UTILITY(state) döndür
    v = -sonsuz
    her a için:
        v = max(v, MIN-VALUE(RESULT(state,a), alpha, beta))
        eğer v >= beta ise v döndür          # beta cutoff
        alpha = max(alpha, v)
    v döndür

MIN-VALUE(state, alpha, beta):
    eğer terminal ise UTILITY(state) döndür
    v = +sonsuz
    her a için:
        v = min(v, MAX-VALUE(RESULT(state,a), alpha, beta))
        eğer v <= alpha ise v döndür        # alpha cutoff
        beta = min(beta, v)
    v döndür

```

Hamle sıralaması

Alpha-Beta'nın etkinliği çocukların hangi sırayla incelendiğine güçlü biçimde bağlıdır. En iyi hamlelerin önce denenebildiği ideal durumda incelenen düğüm sayısı yaklaşık:

$$O(b^{m/2})$$

olur. Böylece etkin dallanma faktörü b yerine yaklaşık $\sqrt{b}$ 'ye düşer. Satrançta $b \approx 35$ ise $\sqrt{35} \approx 6$ 'dır. Bu, aynı sürede yaklaşık iki kat arama derinliğine ulaşabilmek anlamına gelir. Hamleler rastgele sırada incelenirse beklenen davranış idealden kötüdür ancak saf min-max'tan yine daha iyidir. Satrançta taş alma, tehdit, ileri hamle gibi öncelik kuralları move ordering'i iyileştirir.

Iterative deepening, killer move ve transposition table

Yinelemeli derinleştirme (*iterative deepening*), önce 1 ply, sonra 2 ply, sonra 3 ply gibi giderek daha derin arama yapar. Önceki turda iyi bulunan hamleler sonraki turda önce denenerek budama artar. Geçmişte güçlü çıkmış hamleleri önce deneme yaklaşımına **killer move heuristic** (*öldürücü hamle sezgiseli*) denir.

Farklı hamle sıraları aynı oyun konumuna ulaşabiliyorsa **transpozisyon** (*transposition*) oluşur. Aynı konumu tekrar tekrar hesaplamamak için sonuçlar bir hash tablosunda saklanır. Bu tabloya **transposition table** (*transpozisyon tablosu*) denir.

5.4 Eksik Gerçek Zamanlı Kararlar

Minimax oyunun sonuna kadar gitmek ister; Alpha-Beta bazı dalları atsa bile araştırdığı dallarda terminal durumlara ulaşmayı hedefler. Gerçek oyunlarda hamle süresi sınırlı olduğundan bu çoğu zaman mümkün değildir. Claude Shannon'ın klasik önerisi, aramayı daha erken kesmek ve terminal olmayan konumu sezgisel bir değerlendirme fonksiyonuyla tahmin etmektir.

Heuristic Minimax

İki değişiklik yapılır:

1. *UTILITY* yerine konumun değerini tahmin eden *EVAL* kullanılır.
2. *TERMINAL-TEST* yerine ne zaman değerlendirme yapılacağını belirleyen *CUTOFF-TEST* kullanılır.

$$H-MINIMAX(s, d) = \begin{cases} EVAL(s), & CUTOFF-TEST(s, d) \\ \max_a H-MINIMAX(RESULT(s, a), d + 1), & PLAYER(s) = MAX \\ \min_a H-MINIMAX(RESULT(s, a), d + 1), & PLAYER(s) = MIN \end{cases}$$

5.4.1 Değerlendirme fonksiyonları

İyi bir değerlendirme fonksiyonu üç temel özelliğe sahip olmalıdır:

1. Terminal durumları gerçek utility ile aynı sırada değerlendirmelidir: kazanç > beraberlik > kayıp.
2. Çok hızlı hesaplanmalıdır; aksi halde aramayı erken kesmenin kazancı kaybolur.
3. Terminal olmayan durumlarda gerçek kazanma olasılığıyla güçlü biçimde ilişkili olmalıdır.

Satranç deterministik olmasına rağmen arama erken kesildiğinde sonucun ne olacağı hesaplama sınırı nedeniyle bilinmez. Bu yüzden “kazanma olasılığı” dili, oyunun rastlantısından değil, algoritmanın hesaplayamadığı gelecekte doğan belirsizliği ifade eder.

Beklenen değer örneği

Bir pozisyon sınıfının geçmişte %72 kazanma, %20 kaybetme ve %8 beraberlikle sonuçlandığını düşünelim. Kazanç 1, kayıp 0, beraberlik 1/2 ise:

$$E = 0.72(1) + 0.20(0) + 0.08(1/2) = 0.76.$$

Bu değer, o sınıfa ait pozisyonların makul bir değerlendirmesi olabilir.

Taş değeri ve ağırlıklı doğrusal fonksiyon

Satrançta basit bir değerlendirme fonksiyonu taş sayılarına değer atayabilir: piyon 1, at/fil 3, kale 5, vezir 9. Daha genel biçimde:

$$EVAL(s) = \sum_{i=1}^n w_i f_i(s)$$

Burada $f_i(s)$ durum özelliği, w_i ise ağırlığıdır. Şah güvenliği, piyon yapısı ve hareket alanı gibi özellikler de eklenebilir. Doğrusal model her özelliğin katkısının bağımsız olduğunu varsaydığı için pratik programlar etkileşimleri ve doğrusal olmayan kombinasyonları da kullanır.

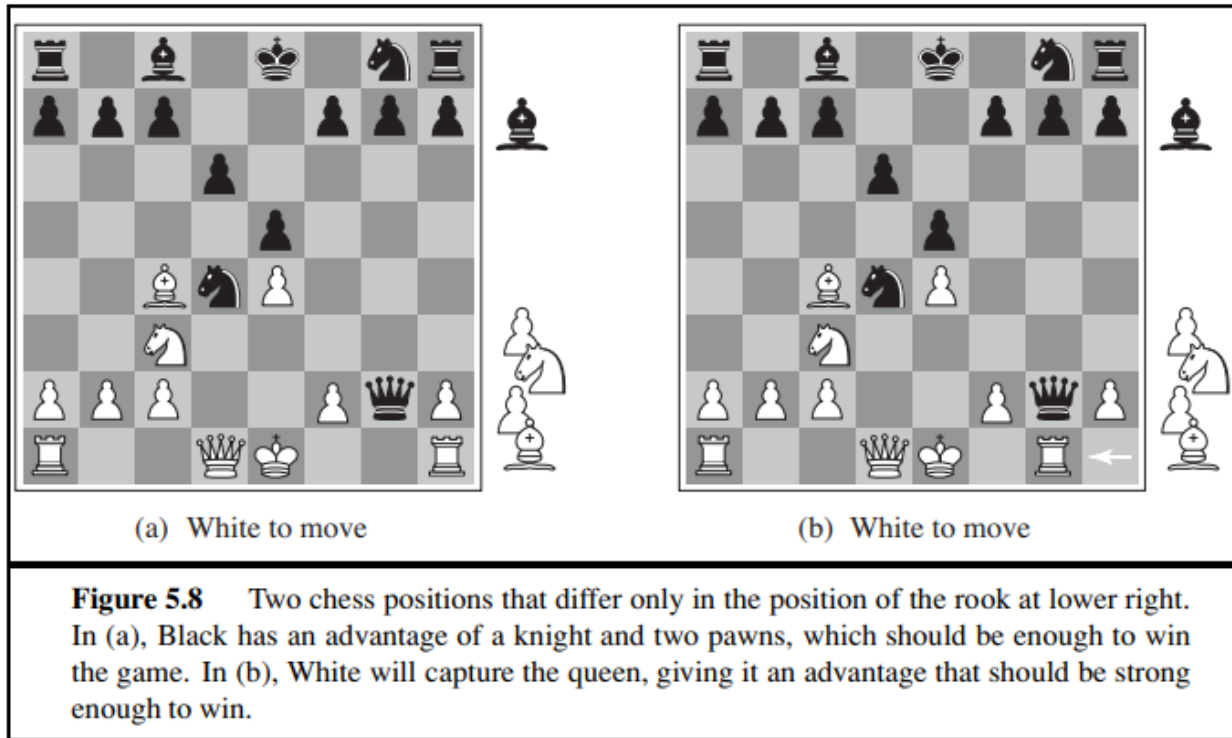


Figure 5.8 Two chess positions that differ only in the position of the rook at lower right. In (a), Black has an advantage of a knight and two pawns, which should be enough to win the game. In (b), White will capture the queen, giving it an advantage that should be strong enough to win.

Şekil 6: Yalnızca sağ alttaki kalenin konumu farklı olan iki satranç pozisyonu. (a)'da Siyah'ın materyal avantajı güçlüdür. (b)'de Beyaz hemen veziri alabileceği için pozisyonun gerçek değeri tersine döner.

Ders mesajı

Bir konumun değeri yalnızca “kaç taş var?” sorusuyla ölçülemez. Taktik tehditler, taşların koordinasyonu, şah güvenliği ve yakın gelecekteki zorunlu hamleler sonucu dramatik biçimde değiştirebilir.

5.4.2 Aramayı kesme

En basit cutoff yaklaşımı sabit bir derinlik d belirlemektir. Daha sağlam yaklaşım iterative deepening kullanmaktır: süre dolduğunda tamamlanmış en derin aramanın sonucu oynanır.

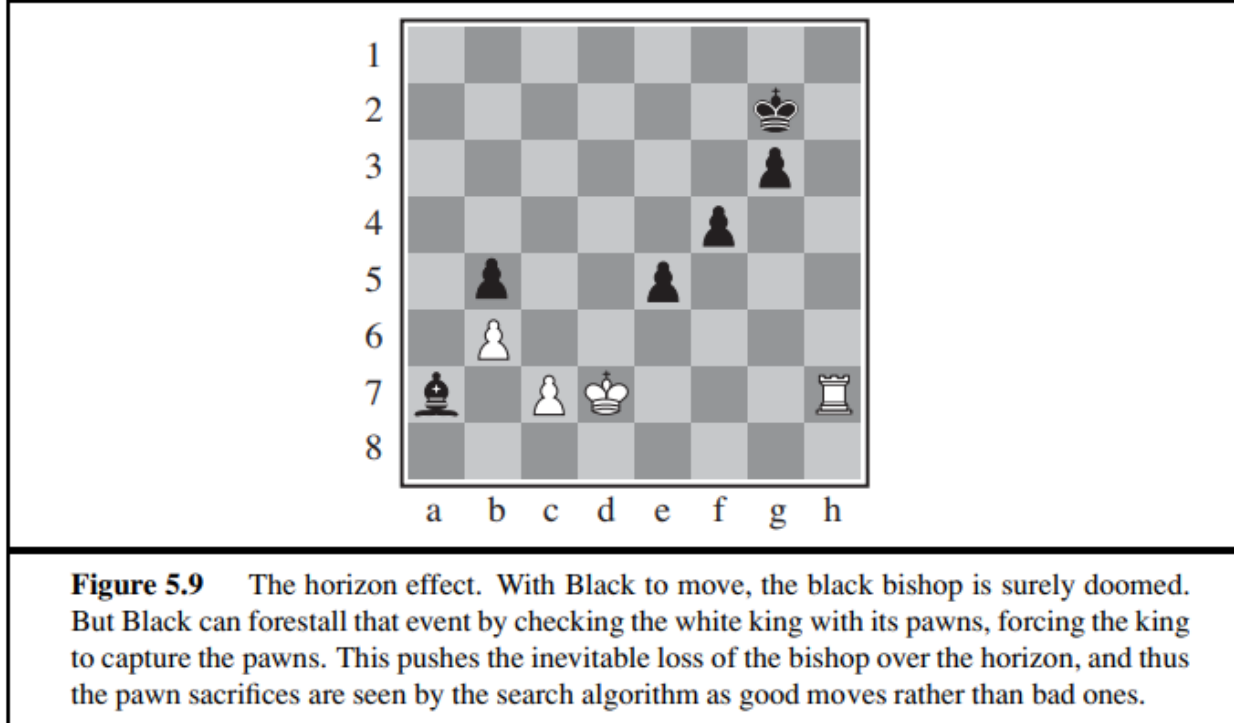
Sabit derinlik, değerlendirme fonksiyonunu taktik açıdan kararsız bir pozisyonda çalıştırabilir. Örneğin materyal sayan bir fonksiyon, bir sonraki hamlede vezirin kaybedileceği konumu yanlış değerlendirebilir.

Quiescence search

Sakin pozisyon (*quiescent position*), yakın gelecekte değerlendirme değerinin büyük sıçrama yapmasının beklenmediği pozisyonudur. Taş alma veya zorlayıcı şah çekme gibi taktikler varsa normal derinlik sınırının biraz ötesine gidilir. Bu ek aramaya **quiescence search** (*sakinlik araması*) denir.

Horizon effect

Ufuk etkisi (*horizon effect*), kaçınılmaz kötü bir olayın çeşitli geciktirme hamleleriyle arama sınırının ötesine itilmesi ve algoritmanın bu gecikmeyi gerçek bir kurtuluş sanmasıdır.



Şekil 7: Ufuk etkisi. Siyah filin kaybı kaçınılmazdır; ancak piyonlarla art arda şah çekmek, filin kaybını sınırlı aramanın ufkunun ötesine geciktirebilir.

Şekilde Siyah piyonlarını feda ederek Beyaz şahı birkaç hamle meşgul eder. Sığ arama, iki piyon karşılığında filin kurtulduğunu sanabilir; oysa daha derine gidildiğinde fil yine kaybedilecektir. **Singular extension** (*tekil uzatma*), belirli bir konumda diğerlerinden açıkça daha önemli görünen zorlayıcı hamleler varsa normal sınırın ötesine seçici biçimde devam ederek bu etkiyi azaltmaya çalışır.

5.4.3 Forward pruning

Alpha-Beta'nın budaması güvenlidir; budanan dalın minimax kararını değiştiremeyeceği kanıtlanır. **Forward pruning** (*ileri budama*) ise bazı hamleleri "muhtemelen kötü" oldukları için daha baştan eler ve bu yüzden en iyi hamleyi yanlışlıkla atma riski taşır.

Beam search (*ışın araması*), her seviyede değerlendirme fonksiyonuna göre yalnızca en iyi görünen n hamleyi tutar. **ProbCut** (,) sığ arama ve geçmiş istatistiklerden yararlanarak bir düğümün (α, β) penceresinin dışında olma olasılığı yeterince yüksekse dalı budar.

5.4.4 Search versus lookup

Açılışta her oyunda milyarlarca durumu yeniden aramak yerine daha önce hazırlanmış açılış kitapları kullanılabilir. Oyun sonunda taş sayısı azaldığında birçok pozisyon tamamen çözülebilir ve en iyi hamle doğrudan tablodan okunabilir. Her durumdan en iyi hamleye eşleme bir **policy** (*politika*) oluşturur.

Örneğin şah+fil+at ile yalnız şah (KBNK) oyun sonunda yaklaşık:

$$462 \times 62 \times 61 \times 2 = 3,494,568$$

olası konum bulunur. Terminal mat durumlarından geriye doğru giderek tüm pozisyonları kazanç/kayıp/beraberlik olarak işaretleme işlemine **retrograde search** (*geriye doğru arama*) denir.

5.5 Stokastik Oyunlar

Gerçek hayatta rastlantısal dış olaylar karar sürecini etkileyebilir. Zar gibi rastlantı unsurları içeren oyunlara **stokastik oyunlar** (*stochastic games*) denir. Tavla, beceri ile şans birleştiren klasik örnektir.

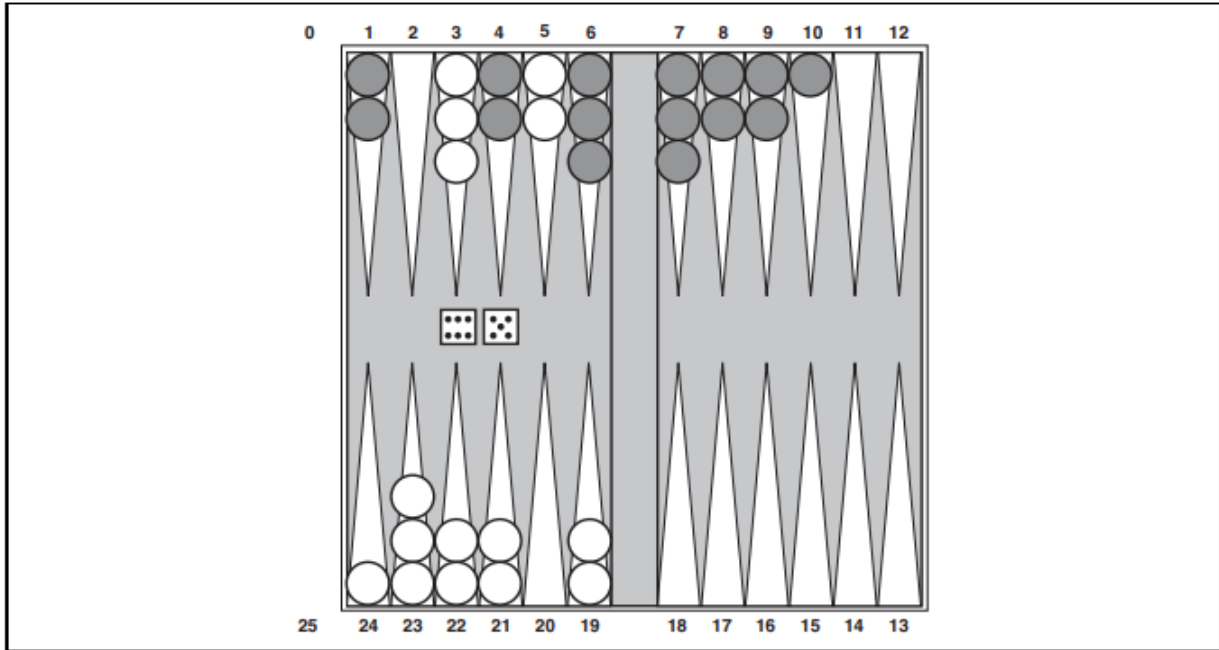


Figure 5.10 A typical backgammon position. The goal of the game is to move all one's pieces off the board. White moves clockwise toward 25, and Black moves counterclockwise toward 0. A piece can move to any position unless multiple opponent pieces are there; if there is one opponent, it is captured and must start over. In the position shown, White has rolled 6-5 and must choose among four legal moves: (5-10,5-11), (5-11,19-24), (5-10,10-16), and (5-11,11-16), where the notation (5-11,11-16) means move one piece from position 5 to 11, and then move a piece from 11 to 16.

Şekil 8: Tipik bir tavla pozisyonu. Beyaz 6-5 atmıştır ve zarın izin verdiği yasal hamlelerden birini seçmek zorundadır.

Satrançta hamle kümesi mevcut tahtadan kesin olarak belirlenir. Tavlada ise oyuncu kendi hamlesini bilse bile rakibin bir sonraki zar sonucunu bilmez. Bu nedenle oyun ağacında MAX ve MIN düğümlerine ek olarak **chance node** (şans düğümü) gerekir.

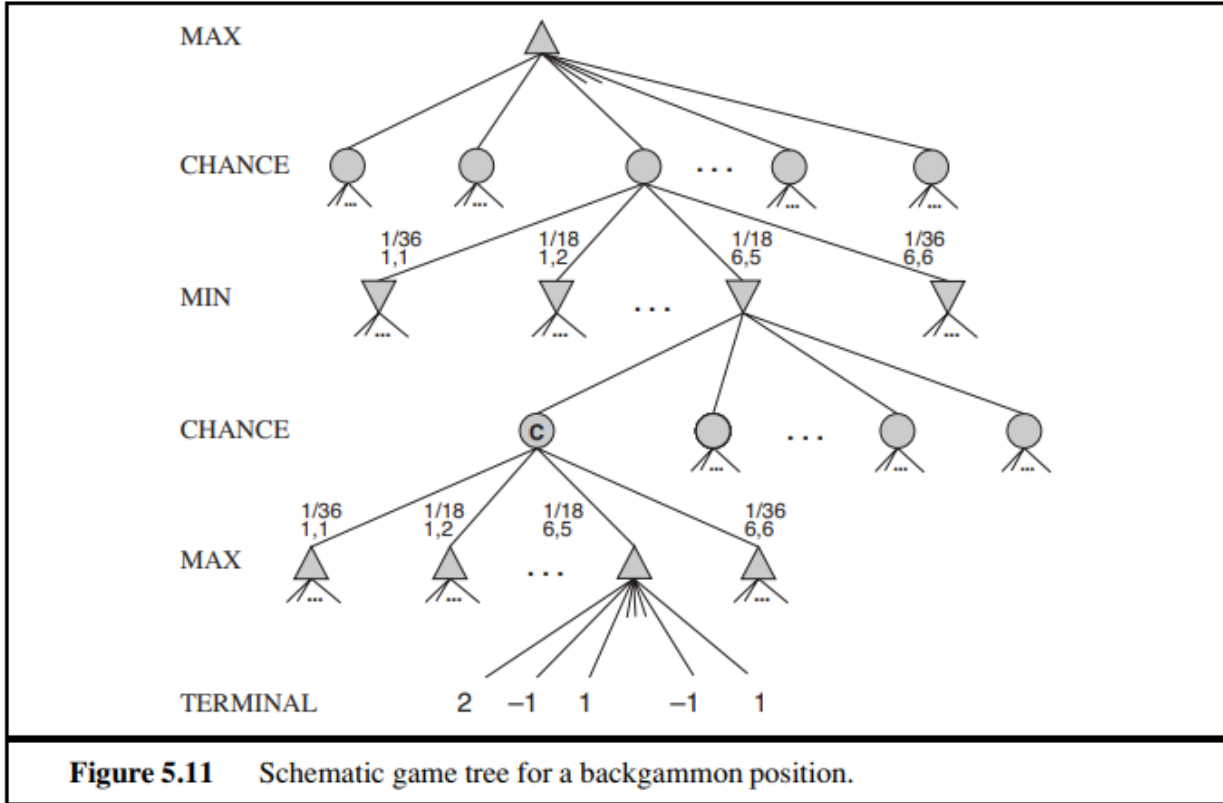


Figure 5.11 Schematic game tree for a backgammon position.

Şekil 9: Tavla için şematik oyun ağacı. Daireler şans düğümleridir; dallarda zar sonucu ve olasılığı bulunur.

İki zar için 36 eş olasılıklı sıralı sonuç vardır. Çiftlerin her biri 1/36, örneğin 1-2 gibi iki farklı sırayla gelebilen sonuçlar ise $2/36 = 1/18$ olasılığa sahiptir.

Expectiminimax

Şans düğümünde maksimum veya minimum alınmaz; olasılık ağırlıklı beklenen değer hesaplanır:

$$EXPECTIMINIMAX(s) = \begin{cases} UTILITY(s), & \text{terminal} \\ \max_a V(RESULT(s, a)), & MAX \\ \min_a V(RESULT(s, a)), & MIN \\ \sum_r P(r) V(RESULT(s, r)), & CHANCE \end{cases}$$

Örnek - Şans düğümü

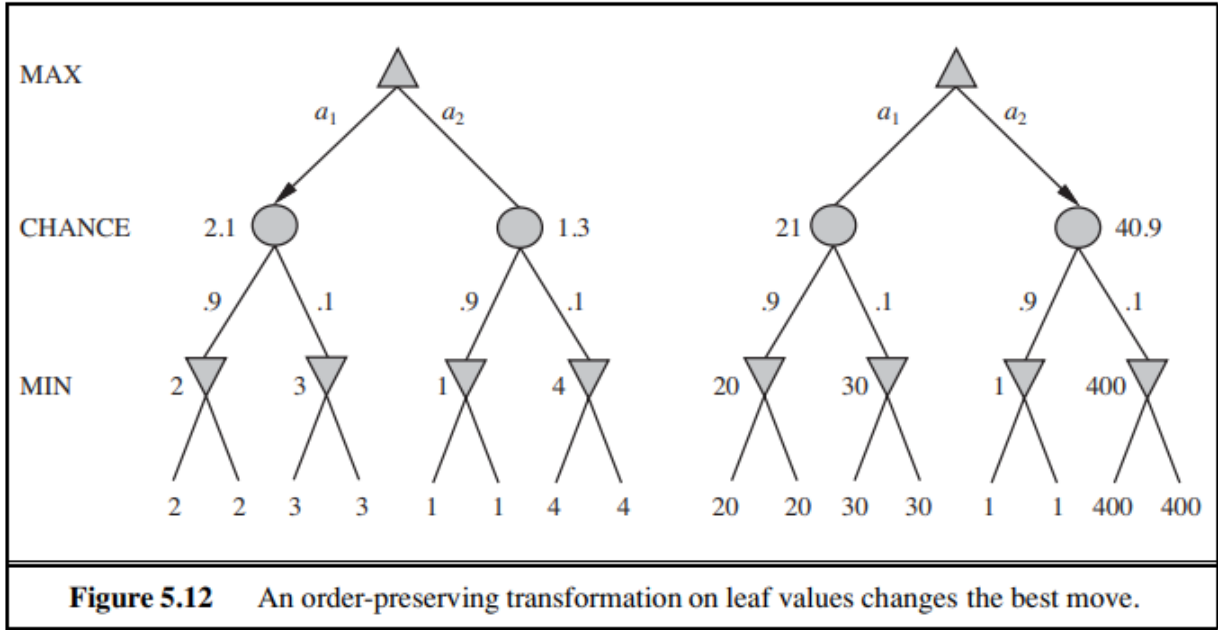
İki olası sonuç 10 ve 4 olsun. Olasılıklar sırasıyla 0.25 ve 0.75 ise:

$$E = 0.25(10) + 0.75(4) = 5.5.$$

Şans düğümünün değeri 5.5 olur.

5.5.1 Şans oyunlarında değerlendirme fonksiyonları

Şans düğümlerinde değerlerin büyüklüğü gerçekten önemlidir; yalnızca sıralamayı korumak yeterli değildir.



Şekil 10: Yaprak değerlerine yalnızca sıralamayı koruyan doğrusal olmayan bir dönüşüm uygulanması, şans düğümleri bulunan ağacın en iyi hamlesini değiştirebilir.

Soldaki değerlerde:

$$0.9(2) + 0.1(3) = 2.1, \quad 0.9(1) + 0.1(4) = 1.3,$$

yani a_1 seçilir. Değerler sıralamayı koruyarak 1, 20, 30, 400 yapılırsa:

$$0.9(20) + 0.1(30) = 21, \quad 0.9(1) + 0.1(400) = 40.9,$$

ve bu kez a_2 seçilir. Bu nedenle şans bulunan oyunlarda değerlendirme ölçeği beklenen utility ile pozitif doğrusal ilişkiyi korumalıdır.

Expectiminimax, rastlantı sonuçlarının tüm dizilerini de dallandırdığı için minimax'tan çok daha hızlı büyüyebilir. Zar sonucu çeşit sayısı n ise kaba maliyet $O(b^m n^m)$ mertebesine çıkabilir. Bu nedenle şans oyunlarında derin arama pahalıdır.

Chance pruning ve Monte Carlo rollout

Utility değerlerinin bir aralıkta olduğu biliniyorsa, bütün çocuklar görülmeden bir chance node için alt/üst sınır hesaplanabilir ve budama yapılabilir. Bir başka yaklaşım, mevcut pozisyondan çok sayıda rastgele oyun simüle etmek ve kazanma oranını pozisyon değeri

olarak kullanmaktır. Zar oyunlarında bu simülasyonlara **rollout** (*oyun açılımı/simülasyonu*) denir.

5.6 Kısmen Gözlemlenebilir Oyunlar

Satranç tam gözlemlenebilirdir; her taşın konumu bilinir. Gerçek çatışmalarda ve birçok oyunda ise rakibin durumu kısmen gizlidir. Böyle ortamlarda ajan tek bir gerçek durumla değil, gözlemlerle uyumlu olası durumların kümesiyle çalışır.

5.6.1 Kriegspiel: kısmen gözlemlenebilir satranç

Kriegspiel'de oyuncular yalnızca kendi taşlarını görür. Hakem bütün taşları bilir. Oyuncu bir hamle önerir; gizli rakip taşları yüzünden hamle yasal değilse hakem “illegal” der. Yasal hamle olduğunda taş alma veya şah bilgisi gibi sınırlı duyurular yapılır.

Bir oyuncunun bugüne kadar aldığı bütün algılarla uyumlu olası tahta durumlarının kümesine **belief state** (*inanç durumu*) denir. Strateji artık rakibin gerçek hamlesine göre değil, oyuncunun alabileceği olası algı dizilerine göre tanımlanır.

Garantili, olasılıksal ve tesadüfi mat

Guaranteed checkmate (*garantili mat*), mevcut belief state içindeki bütün gerçek tahlalarda ve rakibin tüm hamlelerine rağmen mat sağlayan stratejidir.

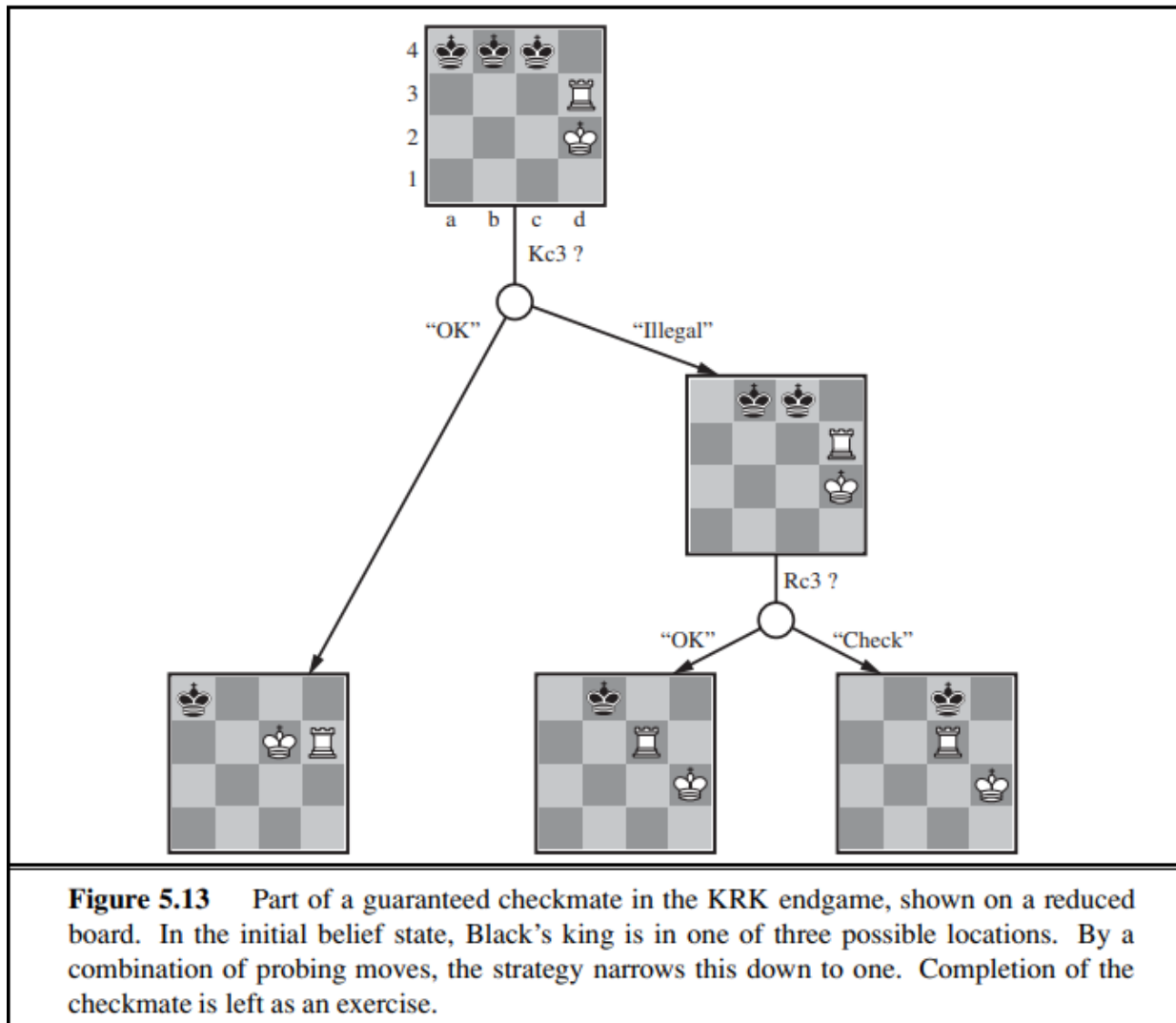


Figure 5.13 Part of a guaranteed checkmate in the KRK endgame, shown on a reduced board. In the initial belief state, Black's king is in one of three possible locations. By a combination of probing moves, the strategy narrows this down to one. Completion of the checkmate is left as an exercise.

Şekil 11: KRK oyun sonunda garantili mat stratejisinin bir bölümü. Beyaz, yoklama hamlelerinden aldığı "OK", "Illegal" ve "Check" geri bildirimleriyle Siyah şahın olası konumlarını daraltır.

Bu figürde bazı hamlelerin amacı doğrudan taş kazanmak değil, bilgi toplamaktır. Kısmen gözlemlenebilir karar problemlerinde bilginin kendisi bir değere sahiptir.

Probabilistic checkmate (*olasılıksal mat*), rastgeleleştirilmiş stratejiyle başarı olasılığının 1'e yaklaşabildiği durumları ifade eder. **Accidental checkmate** (*tesadüfi mat*) ise stratejinin belief state içindeki yalnızca bazı gerçek konumlarda mat etmesi ve oyuncunun gerçek konumu bilmeden matı tesadüfen gerçekleştirmesidir.

Kısmen gözlemlenebilir oyunlarda tamamen tahmin edilebilir bir "optimal" hamle dizisi rakibe bilgi verebilir. Bu nedenle optimal davranış bazen bilinçli rastgelelik içerebilir. Her oyuncu için optimal rastgele stratejilerin birlikte tanımlandığı çözüm kavramı oyun teorisindeki **denge** (*equilibrium*) fikrine götürür.

5.6.2 Kart oyunları

Bridge, whist, hearts ve poker gibi kart oyunları hem rastlantısallık hem de eksik bilgi içerir. Gizli kartların tüm olası dağılımlarını düşünüp her dağılımı tam bilgili oyunmuş gibi

çözmek sezgisel bir yaklaşımdır:

$$\arg \max_a \sum_s P(s) \text{MINIMAX}(\text{RESULT}(s, a)).$$

Bridge’de görünmeyen 26 kartın iki ele 13’er kart dağıtılması:

$$\binom{26}{13} = 10,400,600$$

olası dağılım üretir. Hepsini çözmek pahalı olduğundan N örneklik Monte Carlo yaklaşımı kullanılabilir:

$$\arg \max_a \frac{1}{N} \sum_{i=1}^N \text{MINIMAX}(\text{RESULT}(s_i, a)).$$

Averaging over clairvoyance problemi

Her olası gizli durumu “biraz sonra zaten tamamen görecekmışız” varsayımıyla çözmek bilgi toplama eylemlerinin değerini yok sayar. Bu yaklaşım bilgi saklamayı, takım arkadaşına bilgi aktarmayı veya pokerde blöf yapmayı doğru modelleyemez.

Örnek - Altın ve otobüs benzetmesi

Bir yolda kesin fakat küçük bir ödül, diğer yolda ise hangi kolun ölümcül olduğunu bilmediğiniz büyük bir ödül olsun. Eğer algoritma gelecekte doğru kolu otomatik olarak öğreneceğini varsayarsa riskli yolu gereğinden fazla tercih eder. Gerçek karar verici ise gelecekteki bilgisizliğini de hesaba katmalıdır.

5.7 Döneminin En Gelişmiş Oyun Programları

Bu bölümdeki örnekler kaynak kitabın yazıldığı dönemin tarihsel görünümünü yansıtır. Amaç güncel sıralama vermek değil, güçlü oyun programlarının hangi fikirleri birleştirdiğini göstermektir.

Satranç

IBM Deep Blue, Garry Kasparov’a karşı başarısıyla tanınır. Paralel donanım, özel satranç işlemcileri, Alpha-Beta araması, çok büyük özellikli değerlendirme fonksiyonu, açılış kitabı ve oyun sonu tablolarını birlikte kullanmıştır. Normal arama derinliği yaklaşık 14 ply iken zorlayıcı varyantlarda singular extension sayesinde daha derine çıkabilmiştir.

Null move heuristic (,) rakibe başlangıçta fazladan hamle verilmiş siğ bir aramayla konum için hızlı alt sınır üretmeyi; **futility pruning** (i)se sonucu değiştirmesi beklenmeyen hamleleri önceden eleme fikrini kullanır. Hydra gibi sonraki sistemler agresif bu dama sayesinde benzer hesap gücüyle daha derin aramaya ulaşmıştır. Rybka örneği, iyi

ayarlanmış evaluation function'ın da ham arama gücü kadar önemli olabileceğini göstermiştir.

Checkers, Othello, Backgammon

Chinook, Alpha-Beta ile devasa oyun sonu veritabanlarını birleştirmiştir. Logistello, Othello'da insan dünya şampiyonunu yenmiş ve ProbCut gibi tekniklerin başarısını göstermiştir. TD-Gammon ise reinforcement learning ile neural network tabanlı değerlendirme fonksiyonu öğrenmiş, yalnızca 2-3 ply aramasına rağmen üst düzey insanlarla rekabet edebilmiştir.

Go

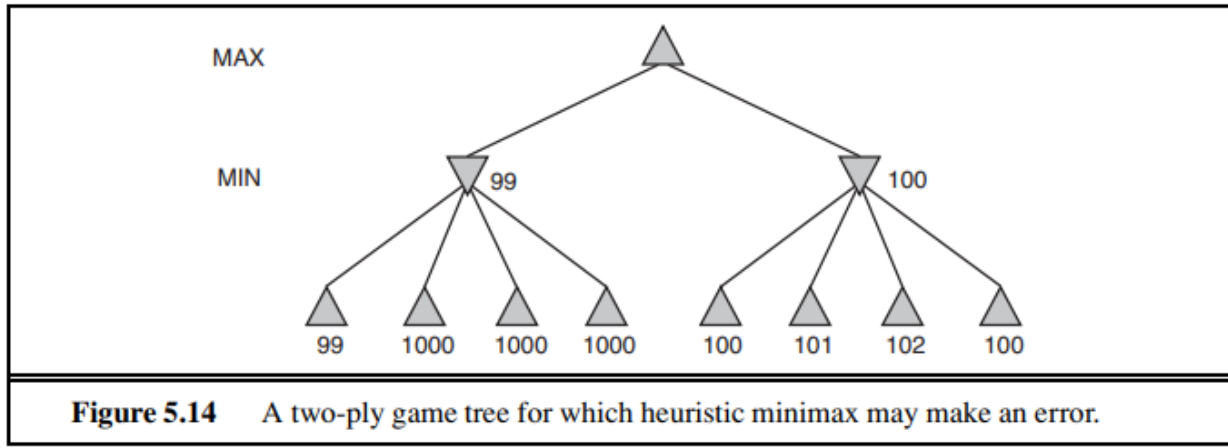
19x19 Go'da başlangıç dallanma faktörü 361'e kadar çıkabildiğinden klasik Alpha-Beta zorlanır. Bu nedenle dönemin güçlü programları Monte Carlo rollouts ve UCT yaklaşımına yönelmiştir. UCT başlangıçta keşif yapar; zamanla geçmiş simülasyonlarda daha çok kazandıran hamleleri daha fazla örnekler. Bu fikir Monte Carlo Tree Search ailesinin merkezindedir.

Bridge ve Scrabble

Bridge Baron hiyerarşik planlar; GIB ise Monte Carlo örnekleme ve açıklama tabanlı genelleştirme kullanmıştır. Scrabble'da en yüksek puanlı kelimeyi bulmak tek başına yeterli değildir; rakibin harfleri ve sonraki çekilişler bilinmediği için oyun hem kısmen gözlemlenebilir hem stokastiktir.

5.8 Alternatif Yaklaşımlar

Minimax + evaluation + Alpha-Beta standart yaklaşım olsa da değerlendirme fonksiyonları yaklaşık olduğu için bu yaklaşım da hatalar yapabilir.



Şekil 12: İki ply'lik ağaçta heuristic minimax'ın hata yapabileceği örnek. Sağ dalın minimumu 100, sol dalın minimumu 99 görünür; ancak değerlendirme hataları hesaba katıldığında sol dalın gerçek değerinin daha yüksek olma ihtimali önemli olabilir.

Şekilde sol MIN düğümü 99, 1000, 1000, 1000; sağ MIN düğümü 100, 101, 102, 100 değerlerine sahiptir. Nominal minimax sağ dalı seçer çünkü $100 > 99$ 'dur. Ancak yaprak tahminlerinin rastgele hataları varsa tek bir düşük tahmin sol dalı haksız yere cezalandırmış olabilir. Komşu düğümlerdeki değerlendirme hatalarının bağımsız olmaması sorunu daha da büyütür.

Düğüm genişletmenin faydası ve metareasoning

Bir konumda yalnızca tek yasal hamle varsa Alpha-Beta yine büyük bir ağaç araştırabilir; oysa hangi hamleyi yapacağımız zaten bellidir. Bu gözlem “Bir düğümü genişletmek karar kalitesini gerçekten artıracak mı?” sorusunu doğurur.

Bir hesaplamanın beklenen karar iyileştirmesi maliyetinden büyükse yapılması mantıklıdır. Hangi hesaplamanın yapılmaya değer olduğuna ilişkin bu üst düzey düşünmeye **metareasoning** (*akıl yürütme hakkında akıl yürütme*) denir. Alpha-Beta bunun en basit örneklerinden biridir: karar üzerinde etkisi olmayacağı ispatlanan hesaplamaları yapmaz.

Hedef yönelimli planlama

İnsan oyuncular çoğu zaman bütün somut hamleleri körlemesine üretmez; “veziri sıkıştır”, “şah kanadını aç”, “geçer piyon oluştur” gibi ara hedefler belirler. Hedef yönelimli planlama, bazı durumlarda kombinatorial aramayı dramatik biçimde azaltabilir. PARADISE programı bu yaklaşımı satranç problemlerinde kullanan erken örneklerden biridir.

5.9 Öğretim Amaçlı Bölüm Özeti

Not

Kullanıcı tarafından sağlanan kaynak parçalarında 5.9'un özgün "Summary" metni bulunmadığı için bu kısım doğrudan çeviri değil, önceki bölümlerin öğretim amaçlı sentezidir.

1. Tam bilgili, deterministik, iki oyunculu sıfır toplamli oyunların temel çözüm modeli Minimax'tır.
2. MAX mümkün olan en yüksek, MIN mümkün olan en düşük değeri seçer.
3. Minimax'ın maliyeti $O(b^m)$ olduğu için büyük oyunlarda tam arama pratik değildir.
4. Alpha-Beta, minimax kararını değiştirmeden gereksiz dalları budar. İdeal move ordering ile maliyet yaklaşık $O(b^{m/2})$ olabilir.
5. Gerçek zamanlı oyunlarda arama belirli bir noktada kesilir ve terminal olmayan durumlar evaluation function ile tahmin edilir.
6. Quiescence search, taktik olarak kararsız durumlarda cutoff hatalarını azaltır; horizon effect, kaçınılmaz kötü olayın arama ufkunun dışına itilmesidir.
7. Rastlantısal oyunlarda chance node ve Expectiminimax kullanılır; chance node değerleri olasılık ağırlıklı ortalamadır.
8. Kısmen gözlemlenebilir oyunlarda tek bir state yerine belief state ve bilgiye bağlı stratejiler gerekir.
9. Monte Carlo yöntemleri büyük, rastlantısal veya kısmen gözlemlenebilir oyunlarda güçlü yaklaşık çözümler sağlayabilir.
10. Güçlü oyun programları genellikle arama, budama, öğrenilmiş/elle tasarlanmış değerlendirme, tablo arama, önbellekleme ve zaman yönetimini birlikte kullanır.

Hızlı formül sayfası

Kavram	Formül / Kural
MAX düğümü	$V(s) = \max_a V(RESULT(s, a))$
MIN düğümü	$V(s) = \min_a V(RESULT(s, a))$
Chance düğümü	$V(s) = \sum_r P(r) V(RESULT(s, r))$
Minimax zaman maliyeti	$O(b^m)$
İdeal Alpha-Beta	$O(b^{m/2})$
Ağırlıklı evaluation	$EVAL(s) = \sum_i w_i f_i(s)$
Chance node kısmi sınır	$S + RL \leq V(C) \leq S + RU$

Pozitif doğrusal dönüşüm $x' = ax + b, a > 0$ karar sırasını korur

Alıştırmalar 5.1-5.22 - Türkçe Çeviri ve Çözümler

Bu bölümde kaynakta verilen alıştırmalar Türkçeye çevrilmiş ve öğretim amaçlı çözümler eklenmiştir. Açık uçlu “uygula” sorularında tek bir doğru program tasarımı olmadığı için örnek tasarım/pseudocode sunulmuştur.

5.1 Rakibin hamlesini bilen kahin

Soru. Her durumda rakibin hamlesini doğru tahmin eden $OM(s)$ kahininiz olduğunu varsayın. Oyunu tek ajanlı bir arama problemi olarak tanımlayın ve optimal hamleyi bulacak algoritmayı açıklayın.

Çözüm - 5.1

MAX a hamlesini yaptıktan sonra durum $s' = RESULT(s, a)$ olsun. Rakibin cevabı artık seçim değil, $OM(s')$ ile bilinen deterministik bir geçiştir. Bu nedenle iki yarım hamlelik birleşik geçiş:

$$RESULT'(s, a) = RESULT(RESULT(s, a), OM(RESULT(s, a)))$$

olarak tanımlanabilir. Böylece MIN düğümleri ortadan kalkar. MAX kendi seçeneklerini tek ajanlı arama gibi araştırır ve terminal utility'si en yüksek yolu seçer.

5.2 İki adet 8-puzzle

Soru özeti. İki 8-puzzle'ı birlikte çözün; ulaşılabilir durum uzayını hesaplayın. Daha sonra oyuncular sırayla hareket etsin, para atışı hangi puzzle üzerinde hamle yapılacağını belirlesin ve bir puzzle'ı ilk çözen kazansın.

Çözüm - 5.2

(a) Durum (P_1, P_2) iki puzzle konfigürasyonundan oluşur. Eylem seçilen puzzle'ın boş karesini yasal yönde hareket ettirir. Hedef, iki puzzle'ın da hedef düzeninde olmasıdır; adım maliyeti 1 alınabilir.

(b) Bir 8-puzzle için ulaşılabilir konfigürasyon sayısı $9!/2 = 181,440$ 'tır. İki bağımsız puzzle için:

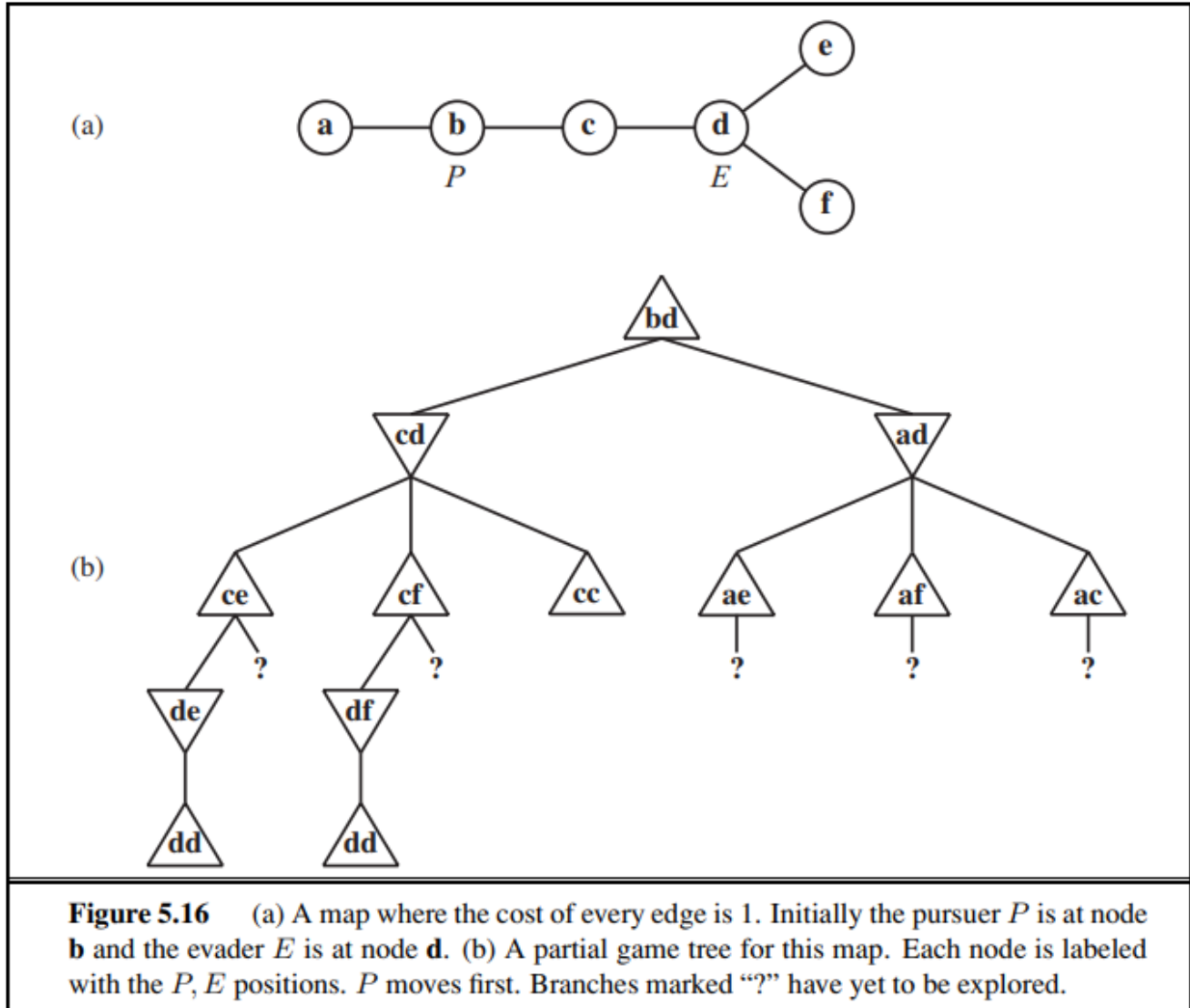
$$\left(\frac{9!}{2}\right)^2 = 32,920,473,600.$$

(c) MAX/MIN kararlarına ek olarak para atışı bulunduğu için **Expectiminimax** kullanılmalıdır.

(d) Durum uzayı sonludur ve her ulaşılabilir puzzle durumundan hedefe sonlu bir çözüm yolu vardır. Para atışının herhangi bir sonlu dizisi pozitif olasılığa sahiptir. Oyunculardan birinin bir puzzle üzerinde yeterince elverişli hamle fırsatı elde ettiği sonlu diziler tekrar tekrar denenebileceğinden, sonsuz oyunda hiçbir oyuncu-

nun kazanamama olasılığı sıfıra gider. Bu, informal olarak bir oyuncunun sonunda kazanacağını destekler.

5.3 Takip-kaçış oyunu



Şekil 13: Her kenar maliyetinin 1 olduğu takip-kaçış haritası ve kısmi oyun ağacı. Düğüm etiketi önce takipçi P , sonra kaçan E konumunu gösterir.

Soru özeti. Takipçi ve kaçan sırayla hareket eder. Aynı düğüme geldiklerinde oyun biter. Takipçinin terminal faydası toplam sürenin negatiftir. Şekildeki kısmi ağacı sınırlar ve shortest-path bilgisiyle analiz edin.

Çözüm - 5.3

Kök bd 'dir; ilk hamleyi takipçi yapar. cc terminali iki hamlede yakalama olduğundan $U(cc) = -2$ 'dir. $bd \rightarrow cd \rightarrow ce \rightarrow de \rightarrow dd$ yolu 4 hamle olduğundan ilgili dd yaprağı -4 'tür; $df \rightarrow dd$ için de aynı değer elde edilir.

$de = df = -4$. ce ve cf MAX düğümleri oldukları için en az -4 değerindedir. cd MIN düğümünde çocuklardan biri -2 olduğu için $V(cd) \leq -2$.

Soru işaretli dalların durumları sırasıyla be , bf , be , bf , bc 'dir. Bu düğümlere gelindiğinde 3 hamle harcanmıştır. Haritada $d(b, e) = 3$, $d(b, f) = 3$, $d(b, c) = 1$ olduğundan en erken yakalama süreleri 6, 6 ve 4'tür. Utility negatif süre olduğuna göre:

$$V(be) \leq -6, \quad V(bf) \leq -6, \quad V(bc) \leq -4.$$

Böylece $ce = cf = -4$ ve $cd = -4$ bulunur. Sağdaki ad dalında ilk çocuk bile ≤ -6 sınırı verdiğinde kökteki -4 alternatifi nedeniyle geri kalan sağ dallar budanabilir. Kök değeri -4 ve takipçinin optimal ilk hamlesi $b \rightarrow c$ olur.

Sonlu ve ağaç biçimindeki bir haritada çevrim yoktur. Takipçi kaçana giden tek yol boyunca ilerlediğinde kaçan sonsuza kadar dolaşamaz ve bir yaprakta sıkışır; bu nedenle takipçi sonunda kazanır.

5.4 Stokastik oyun modeli tasarlama

Soru. Monopoly, Scrabble, belirli kontratlı Bridge veya Texas Hold'em için state, move generator, terminal test, utility ve evaluation function tanımlayın.

Çözüm - 5.4 - Monopoly örneği

Durum; oyuncu konumları, nakitler, mülk sahiplikleri, binalar, sıra, hapisane durumu ve zar bilgisini içerir. Move generator zar sonucuna ve kurallara göre hareket, satın alma, bina, ipotek gibi seçenekleri üretir. Terminal test tek bir iflas etmemiş oyuncu kaldığında true olabilir. İki oyunculu basit utility $+1/-1$ alınabilir. Evaluation için net servet farkı, nakit farkı, tam renk setleri ve bina değerleri ağırlıklı toplam olarak kullanılabilir. Zar nedeniyle Expectiminimax gerekir.

5.5 Gerçek zamanlı çok oyunculu ortam

Çözüm - 5.5

Zamanı durumun parçası yapın:

$$s = (board, player, t_1, t_2, \dots).$$

Oyuncu bir hamleyi Δt sürede hesaplarsa kalan süresi $t'_i = t_i - \Delta t$ olur. Süre sıfıra düşerse oyuncu zaman aşımından kaybeder. Bu ortam, ajanı yalnızca iyi hamle seçmeye değil, düşünme süresini yönetmeye de zorlar.

5.6 Sürekli fiziksel oyunlar

Çözüm - 5.6

Tenis, bilardo ve kroket gibi oyunlarda durum ve eylem uzayı sürekli. Bir bilardo vuruşu açı, kuvvet, temas noktası ve falso gibi sürekli parametrelerden oluşur. Bu nedenle standart ayrık Minimax doğrudan uygulanamaz. Pratik yaklaşım; hareket uzayını örneklemek, fizik simülasyonu kullanmak, olası vuruşları sınırlı sayıda üretmek ve Monte Carlo/trajectory optimization gibi yöntemleri aramayla birleştirmektir.

5.7 Suboptimal rakibe karşı minimax

Çözüm - 5.7

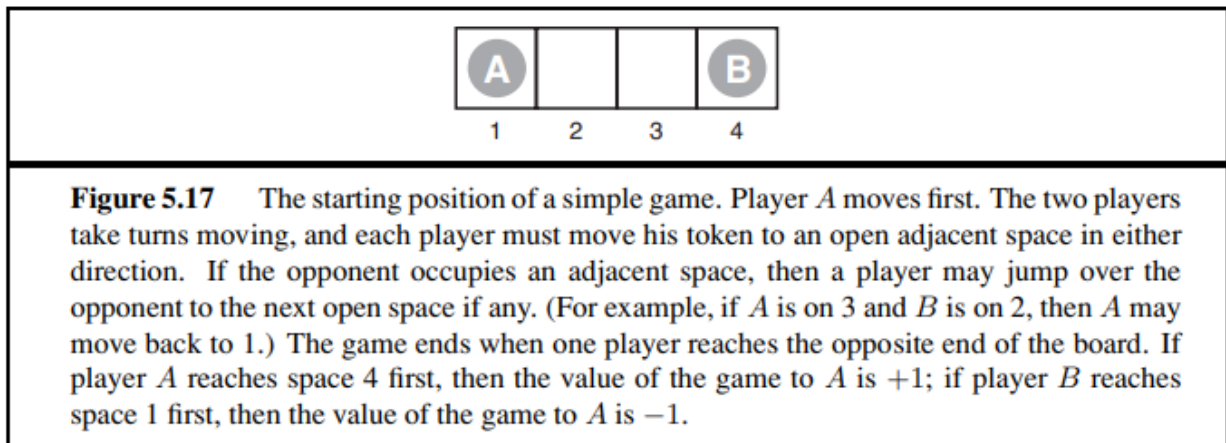
MAX'in minimax stratejisi σ^* ve oyun değeri V olsun:

$$V = \min_{\tau} U(\sigma^*, \tau).$$

Bu tanım gereği herhangi bir MIN stratejisi için $U(\sigma^*, \tau) \geq V$ 'dir. Dolayısıyla suboptimal MIN, MAX'in sonucunu optimal MIN'den daha aşağı indiremez.

Ancak MAX rakibin özel hatalarını biliyorsa minimax dışı bir stratejiyle daha çok kazanabilir. Örneğin bir dal garantili 5, başka dal optimal MIN'e karşı 0 ama hatalı MIN'e karşı 100 veriyorsa, hatalı rakibe karşı ikinci dalı seçmek daha kârlıdır.

5.8 Dört karelik oyun ve döngüler



Şekil 14: Dört karelik oyunun başlangıç durumu. A önce oynar; rakip komşu karedeyse arkasındaki boş kareye atlama yapılabilir.

Durum (s_A, s_B) biçiminde yazılır. Başlangıç $(1, 4)$ 'tür. Zorunlu başlangıç dizisi $(1, 4) \rightarrow (2, 4) \rightarrow (2, 3)$ 'tür. Buradan A, B'nin üzerinden atlayıp $(4, 3)$ durumuna giderek hemen +1 kazanabilir; geri hareket edilirse döngülü durumlar ortaya çıkar.

Standart Minimax aynı durum tekrar tekrar ziyaret edildiğinde sonsuz recursion'a girebilir. Çözüm, mevcut path üzerindeki tekrar durumlarını tespit etmek, oyun ağacı yerine oyun grafi üzerinde çalışmak ve döngülerin değerini fixed-point/retrograde tarzı yöntemlerle belirlemektir. Döngüyü otomatik olarak beraberlik saymak bütün oyunlar için doğru değildir; sonsuz oyunun utility'si açıkça tanımlanmalıdır.

n karelik genellemede parity sonucu belirler: n çift ise A, n tek ise B kazanır. Küçük taban durumları $n = 3$ ve $n = 4$ ile görülebilir; iki kare eklemek oyunun merkez etkileşiminin parity'sini korur.

5.9 Tic-tac-toe evaluation alıştırması

Tanım. X_n , tam n adet X ve hiç O içermeyen satır/sütun/çapraz sayısı; O_n benzer biçimde O içindir. Terminalde X kazanırsa +1, O kazanırsa -1, diğer terminal durumlarda 0 verilir. Terminal olmayan durumda:

$$Eval(s) = 3X_2(s) + X_1(s) - (3O_2(s) + O_1(s)).$$

Çözüm - 5.9

(a) Tüm hamle sıraları için kaba üst sınır $9! = 362,880$ 'dir. Kazanma oluştuğunda oyun erken bittiği için gerçek yasal oyun dizisi sayısı 255,168'dir.

(b) Simetri altında ilk X hamlesi üç sınıfa ayrılır: merkez, köşe, kenar. İkinci O hamlesiyle toplam 12 simetrik depth-2 durum elde edilir.

(c) Bir X ve bir O bulunduğu için $X_2 = O_2 = 0$, dolayısıyla $Eval = X_1 - O_1$ olur.

İlk X	O konumu	Eval
Merkez	Köşe	1
Merkez	Kenar	2
Köşe	Komşu kenar	1
Köşe	Komşu köşe	0
Köşe	Merkez	-1
Köşe	Uzak kenar	1
Köşe	Karşı köşe	0
Kenar	Komşu köşe	-1
Kenar	Komşu kenar	0
Kenar	Merkez	-2
Kenar	Uzak köşe	-1
Kenar	Karşı kenar	0

(d) MIN katmanında merkez ilk hamlesinin geri taşınmış değeri $\min(1, 2) = 1$; köşenin -1; kenarın -2'dir. Kök MAX bu nedenle merkezi seçer.

(e) Optimal Alpha-Beta sıralamasında merkez dalı önce değerlendirilip $\alpha = 1$ elde edilir. Köşe dalında ilk -1, kenar dalında ilk -2 görüldüğünde kalan çocuklar budanır. Simetri sınıfları üzerinden sayıldığında 8 depth-2 durum değerlendirilmeden bırakılabilir.

5.10 Genelleştirilmiş tic-tac-toe

Çözüm - 5.10

$N = |S|$ olsun. Derinlik k 'daki düğüm sayısı en fazla $N!/(N - k)!$ olduğundan:

$$T(N) \leq \sum_{k=0}^N \frac{N!}{(N - k)!} = N! \sum_{j=0}^N \frac{1}{j!} < eN!.$$

Dolayısıyla $T(N) = O(N!)$. En kötü durumda $W = \emptyset$ ise oyun erken bitmez ve aynı ifade gerçek toplam düğüm sayısını verir; alt sınır $\Omega(N!)$ 'dir.

Genel evaluation için, rakip tarafından bloke edilmemiş her kazanma kümesine işaret sayısı ile üstel artan değer verilebilir:

$$Eval(s) = \sum_{w \in W} \left[I(O \cap w = \emptyset) c^{|X \cap w|} - I(X \cap w = \emptyset) c^{|O \cap w|} \right], \quad c > 1.$$

2 GHz işlemci ve düğüm başına yaklaşık $100N$ instruction ile, ideal move ordering altında Alpha-Beta'nın yaklaşık $\sqrt{eN!}$ düğüm değerlendirdiğini varsayarsak kaba çözüm sınırları: 1 saniyede $N \approx 14$, 1 dakikada $N \approx 17$, 1 saatte $N \approx 20$ civarındadır. Bunlar yaklaşık büyüklük tahminleridir.

5.11 Genel oyun oynayan program

Çözüm - 5.11

Othello örneğinde state $(board, player)$; move generator yasal yerleştirmeleri ve çevrilen taşları üretir. Terminal, iki oyuncunun da yasal hamlesi kalmadığında oluşur. Evaluation örneği:

$$EVAL = w_1(\text{taş farkı}) + w_2(\text{mobility}) + w_3(\text{köşe kontrolü}) - w_4(\text{frontier}).$$

Genel Alpha-Beta agent cutoff derinliğine kadar arar. Derinliği artırmak daha çok taktik görür fakat b^d maliyeti yaratır; move ordering budamayı artırır; daha iyi evaluation aynı derinlikte karar kalitesini yükseltir. Selective search yöntemleri, kararı değiştirmesi en olası dallara daha fazla hesap ayırır.

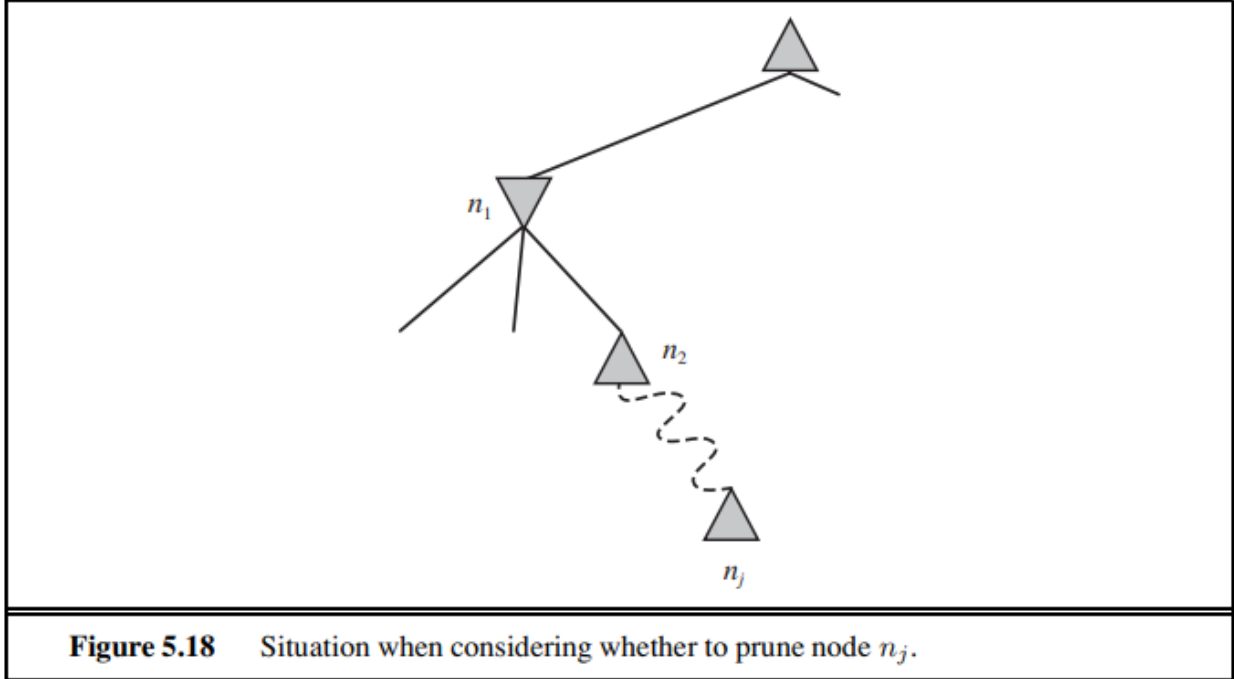
5.12 İki oyunculu sıfır toplamli olmayan oyunlar

Çözüm - 5.12

Her terminal düğüm tek sayı yerine (U_1, U_2) çifti taşır. Oyuncu 1 düğümünde U_1 'i, oyuncu 2 düğümünde U_2 'yi maksimize eden çocuk seçilir. İki utility arasında hiçbir kısıt yoksa bir oyuncunun değeri diğerinin tercihi hakkında sınır vermediği için klasik Alpha-Beta tarzı genel budama mümkün değildir.

Eğer her durumda $|U_1 - U_2| \leq k$ ise bir oyuncunun değeri diğerinin değerine sınır koyar. k küçüldükçe iki oyuncunun çıkarları birbirine yaklaşır ve bu ek sınırlar bazı dalların diğer oyuncu tarafından seçilemeyeceğini kanıtlayarak budama fırsatı yaratır.

5.13 Alpha-Beta doğruluk ispatı



Şekil 15: Alpha-Beta doğruluğunu kanıtlarken n_j düğümünün budanıp budanamayacağını incelediği genel yol.

Çözüm - 5.13

n_1 MIN ise:

$$V(n_1) = \min(l_1, V(n_2), r_1),$$

n_2 MAX ise:

$$V(n_2) = \max(l_2, V(n_3), r_2),$$

ve min/max dönüşümlü olarak devam eder. Burada l_i solda daha önce bilinen kardeşlerin birleşik değeri, r_i sağdaki henüz araştırılmamış kardeşlerin birleşik değeridir.

n_j bir MAX düğümü olsun. Yol üzerindeki MIN atalarının mevcut en iyi sınırlarının minimumu β olsun. $V(n_j) \geq \beta$ olduğu anda üstteki MIN atası zaten β veya daha iyi (MIN için daha düşük) bir alternatifte sahiptir. Dolayısıyla n_j 'nin tam değerinin daha da büyümesi n_1 değerini değiştiremez ve dal budanabilir.

n_j bir MIN düğümü olduğunda simetrik olarak yol üzerindeki MAX atalarının en iyi sınırı α 'dır. $V(n_j) \leq \alpha$ olduğunda MAX bu yolu seçmeyeceği için kalan çocuklar budanır. Böylece Alpha-Beta'nın yalnızca kökün minimax değerinden bağımsız dalları attığı gösterilir.

5.14 Alpha-Beta'nın ideal karmaşıklığı

Çözüm - 5.14

İkili ağaçta optimal move ordering ile yaklaşık her iki seviyede yalnızca bir tam dallanma gerekir. Yaprak sayısı:

$$L(m) = 2^{\lceil m/2 \rceil} + 2^{\lfloor m/2 \rfloor} - 1$$

mertebesindedir. Çift $m = 2k$ için $L = 2^{k+1} - 1 = O(2^k) = O(2^{m/2})$. Genel dallanma faktörü b için aynı argüman $O(b^{m/2})$ sonucunu verir.

5.15 Transposition table kapasitesi

Çözüm - 5.15

Kompakt bir kayıt; 64 bit hash, evaluation, depth, bound type, best move ve meta-data ile yaklaşık 16 byte tutulabilir. 2 GiB bellekte:

$$\frac{2^{31}}{16} = 134,217,728$$

yani yaklaşık 134 milyon entry sığar.

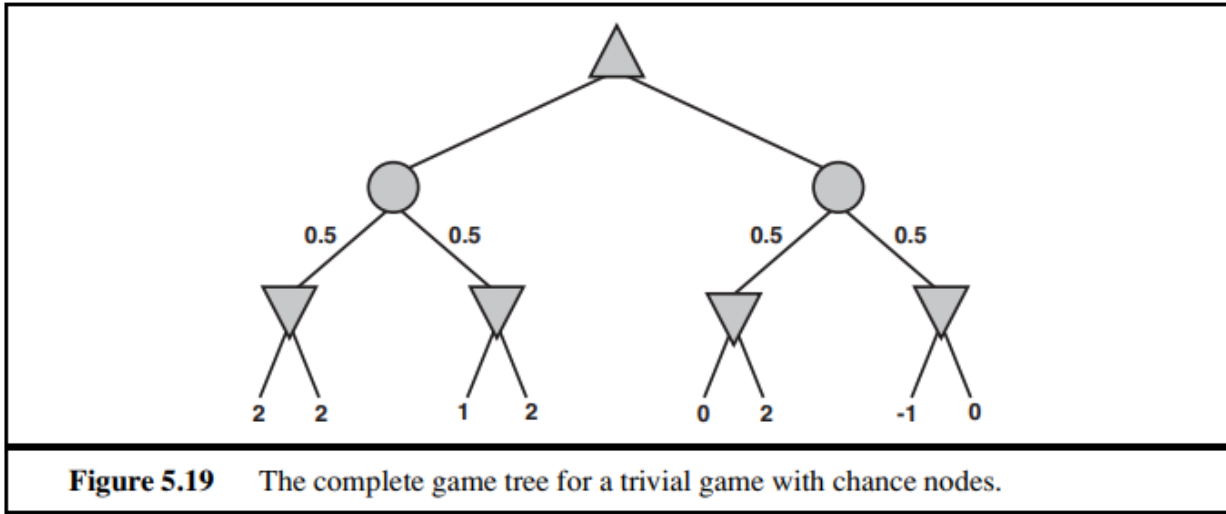
10 milyon node/s hızında 3 dakika arama:

$$10^7 \times 180 = 1.8 \times 10^9$$

node ziyareti demektir; dolayısıyla tüm ziyaretleri aynı anda saklamaya yetmez fakat tekrar durumları ve replacement politikaları sayesinde yine çok değerlidir.

Bir evaluation yaklaşık 100 ns sürer. RAM'deki rastgele lookup donanıma bağlı olarak benzer mertebededir. Mekanik diskte yaklaşık 10 ms seek varsayılırsa bu sürede yaklaşık $10^7 \times 0.01 = 100,000$ evaluation yapılabilir; bu nedenle gerçek zamanlı transposition table RAM'de tutulmalıdır.

5.16 Chance node'larda budama



Şekil 16: Chance node'lu basit oyunun tam ağacı. Yapraklar soldan sağa 2, 2, 1, 2, 0, 2, -1, 0 ve chance olasılıkları 0.5'tir.

Çözüm - 5.16

Alt MIN düğümleri sırasıyla:

$$2, \quad 1, \quad 0, \quad -1$$

değerindedir. Sol chance:

$$0.5(2) + 0.5(1) = 1.5,$$

sağ chance:

$$0.5(0) + 0.5(-1) = -0.5.$$

Kök MAX sol dalı seçer ve değeri 1.5 olur.

İlk altı yaprak biliniyorsa 7. yaprağı değerlendirmek gerekir; çünkü son MIN düğümü sınırsız olabilir. 7. yaprak -1 görüldüğünde o MIN düğümü ≤ -1 olur ve sağ chance node'un üst sınırı -0.5'e düşer; 8. yaprak artık gereksizdir.

Yaprakların $[-2, 2]$ aralığında olduğu biliniyorsa ilk iki yapraktan sonra sol chance node aralığı:

$$0.5(2) + 0.5[-2, 2] = [0, 2]$$

olur. İlk dört yaprak sol chance değerini 1.5 yapar. Sağda yalnızca 5. yaprak 0 görüldüğünde ilk MIN'in üst sınırı 0, ikinci MIN'in üst sınırı 2'dir; chance üst sınırı $0.5(0) + 0.5(2) = 1 < 1.5$ olduğundan 6., 7. ve 8. yapraklar budanabilir.

5.17 Expectiminimax ve *-Alpha-Beta

Çözüm - 5.17

Expectiminimax'ta MAX için max, MIN için min, CHANCE için olasılık ağırlıklı toplam kullanılır. Utility $[L, U]$ aralığında ve bir chance node'da değerlendirilen çocukların katkısı S , henüz görülmeyen olasılık toplamı R ise:

$$S + RL \leq V(C) \leq S + RU.$$

Üst sınır α 'nın altında kaldığında veya alt sınır β 'nin üstüne çıktığında uygun bağlama göre kalan chance dalları budanabilir. Etkinliği, normal expectiminimax düğüm sayısı ile *-Alpha-Beta düğüm sayısını karşılaştırarak ölçebilirsiniz.

5.18 Pozitif doğrusal dönüşüm

Çözüm - 5.18

$x' = ax + b$, $a > 0$ olsun. MAX için:

$$\max_i(ax_i + b) = a \max_i x_i + b;$$

MIN için aynı biçimde min korunur. Chance için:

$$\sum_i p_i(ax_i + b) = a \sum_i p_i x_i + b \sum_i p_i = aE[x] + b$$

çünkü $\sum_i p_i = 1$. Ağacın her düğümünde yeni değer $aV + b$ olduğundan ve $a > 0$ sıralamayı koruduğundan kökte seçilen hamle değişmez.

5.19 Zar dizilerini önceden örnekleme

Çözüm - 5.19

Gelecekteki zar dizisini önceden sabitleyip her örneği Alpha-Beta ile çözmek genel olarak yanlıdır. Çünkü her deterministik çözüm, oyuncuya gelecekte hangi zarların geleceğini biliyormuş gibi davranma fırsatı verir. Gerçek oyuncu bu bilgiye sahip değildir. Bu “clairvoyance” avantajı nedeniyle genel olarak $E[\max X] \neq \max E[X]$ olur. Monte Carlo kullanılabilir fakat karar politikası henüz gözlenmemiş zar sonuçlarına koşullanmamalıdır.

5.20 Max ve Expectimax ağaçlarında budama

Çözüm - 5.20

- (a) Sonlu fakat üstten sınırsız yaprak değerli max tree'de genel budama yoktur; görülmeyen yaprak arbitrarily büyük olabilir.
- (b) Aynı koşulda expectimax'ta da yoktur; sıfırdan büyük olasılıklı görülmeyen sonuç arbitrarily büyük expectation katkısı yapabilir.
- (c) Değerlerin yalnızca nonnegative olması yine üst sınır vermez; max tree'de genel budama yoktur.
- (d) Expectimax için de aynı gerekçe geçerlidir.
- (e) Değerler $[0, 1]$ ise max tree'de budama mümkündür. Bir çocuk 1 bulunduyrsa bundan daha iyi sonuç olamaz.
- (f) Expectimax'ta da mümkündür. Örneğin kökte mevcut alternatif 0.8 ise ve başka chance dalının kalan sonuçlarıyla ulaşabileceği maksimum değer 0.5 olarak sınırlandıysa kalan sonuçlar budanır.
- (g) Yüksek olasılıklı sonuçları önce değerlendirmek genellikle daha iyi budama fırsatı verir; kalan probability mass daha hızlı küçülür ve chance node aralığı daha çabuk daralır.

5.21 Doğru / yanlış

Çözüm - 5.21

- (a) **Doğru.** Tam bilgili, iki rasyonel oyunculu sıfır toplamlı oyunda optimal MIN stratejisini bilmek MAX'ın minimax değerinin üstüne çıkmasını garanti etmez.
- (b) **Yanlış.** Kısmen gözlemlenebilir oyunda rakibin hangi hamleyi yapacağını bilmek gizli state hakkında bilgi sağlayabilir; bilginin kendisi değerlidir.
- (c) **Yanlış.** Kusursuz rasyonel tavla ajanı beklenen utility'yi maksimize eder, ancak zar nedeniyle kötü rastlantı dizileriyle yine kaybedebilir.

5.22 Chance ve eksik bilginin birlikte etkisi

Çözüm - 5.22

Oyun	Chance	Eksik bilgi	Standart Expectiminimax
Monopoly	Evet	Genelde hayır	Uygun
Scrabble	Evet	Evet	Tek başına yetersiz
Bridge	Evet	Evet	Tek başına yetersiz
Texas Hold'em	Evet	Evet	Tek başına yetersiz

Monopoly'de zar chance node olarak doğrudan modellenenebilir. Scrabble, Bridge ve pokerde rakibin gizli kart/harfleri nedeniyle belief state veya information set gerekir. Monte Carlo determinization pratik bir yaklaşık yöntem olabilir; ancak

örneklenmiş gizli bilgiyi gerçek oyuncu biliyormuş gibi kullanmak clairvoyance hatası yaratır. Strateji gerçek state'e değil, oyuncunun gözlem geçmişine ve bu geçmişten türetilen olasılık dağılımına bağlı olmalıdır. Bu sayede bilgi toplama, bilgi saklama ve blöf gibi davranışlar temsil edilebilir.

Kavram Sözlüğü

Terim	Kısa açıklama
Adversarial Search	Rakibin kendi çıkarına göre cevap verdiği karar problemlerindeki arama.
Game Tree	Oyunun teorik tüm hamle dizilerini içeren ağaç.
Search Tree	Algoritmanın gerçekten araştırdığı oyun ağacı bölümü.
MAX / MIN	Faydayı sırasıyla büyüten ve küçülten oyuncu düğümleri.
Ply	Tek oyuncunun tek hamlesi; iki oyuncunun birer hamlesi iki ply'dir.
Minimax	Kusursuz rakibe karşı garanti edilen sonucu maksimize eden algoritma.
Alpha-Beta	Minimax sonucunu değiştirmeden gereksiz dalları budayan yöntem.
Evaluation Function	Terminal olmayan durumun utility'sini tahmin eden fonksiyon.
Cutoff Test	Aramanın hangi noktada durdurulacağını belirleyen test.
Quiescence Search	Taktik açıdan kararsız pozisyonlarda biraz daha derine giden ek arama.
Horizon Effect	Kaçınılmaz olayın sınırlı arama ufkunun dışına itilmesi.
Transposition Table	Daha önce değerlendirilen konumları hash tablosunda saklama.
Chance Node	Rastlantısal sonucu temsil eden düğüm.
Expectiminimax	MAX, MIN ve chance düğümlerini birlikte değerlendiren algoritma.
Rollout	Rastgele gelecekleri simüle ederek konum değerlendirme.
Belief State	Eksik bilgi altında gerçek durum olabilecek durumların kümesi/dağılımı.
Randomized Strategy	Tahmin edilebilirliği azaltmak veya denge sağlamak için olasılıklı strateji.
Metareasoning	Hangi hesaplamayı yapmaya değer olduğuna ilişkin akıl yürütme.

Son tekrar

Deterministik + tam bilgi: Minimax / Alpha-Beta.

Zaman sınırlı: Cutoff + Evaluation + Quiescence.

Rastlantı: Chance node + Expectiminimax / Monte Carlo.

Eksik bilgi: Belief state + bilgiye bağlı strateji.

Çok büyük arama: Move ordering + pruning + transposition + lookup + seçici arama.

Ünite 5 sonu.